

CS 305: Computer Networks

Fall 2025

Network Layer – The Control Plane

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

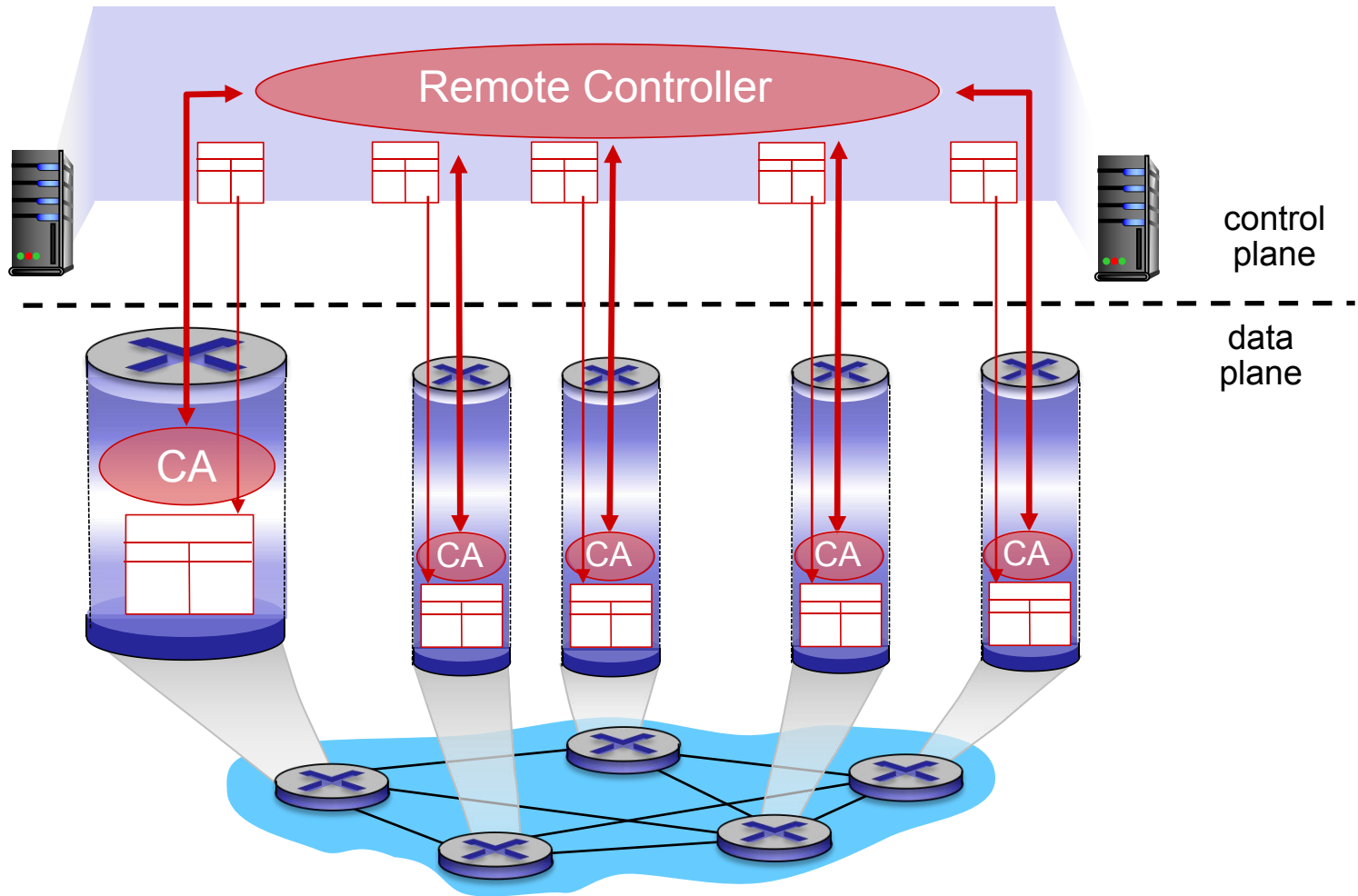
5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

Recall: SDN logically centralized control plane

A distinct (typically remote) controller interacts with local control agents (CAs) in routers to compute forwarding tables



Software defined networking (SDN)

Why a **logically centralized** control plane?

- easier network management: avoid router misconfigurations, greater flexibility of traffic flows
- table-based forwarding (recall OpenFlow API) allows “programming” routers
 - centralized “programming” easier: compute tables centrally and distribute
 - distributed “programming” more difficult: compute tables as result of distributed algorithm (protocol) implemented in each and every router
- open (non-proprietary) implementation of control plane

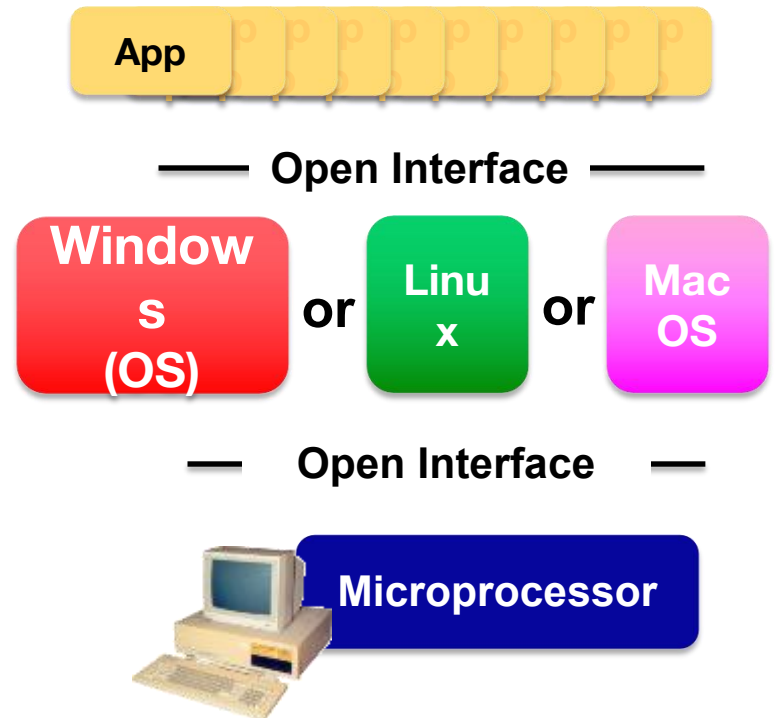
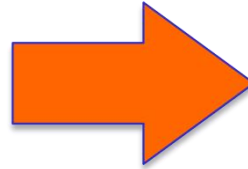
路由器只负责：
按表转发包 (match → action)

开放接口

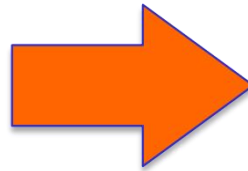
Analogy: mainframe to PC evolution*



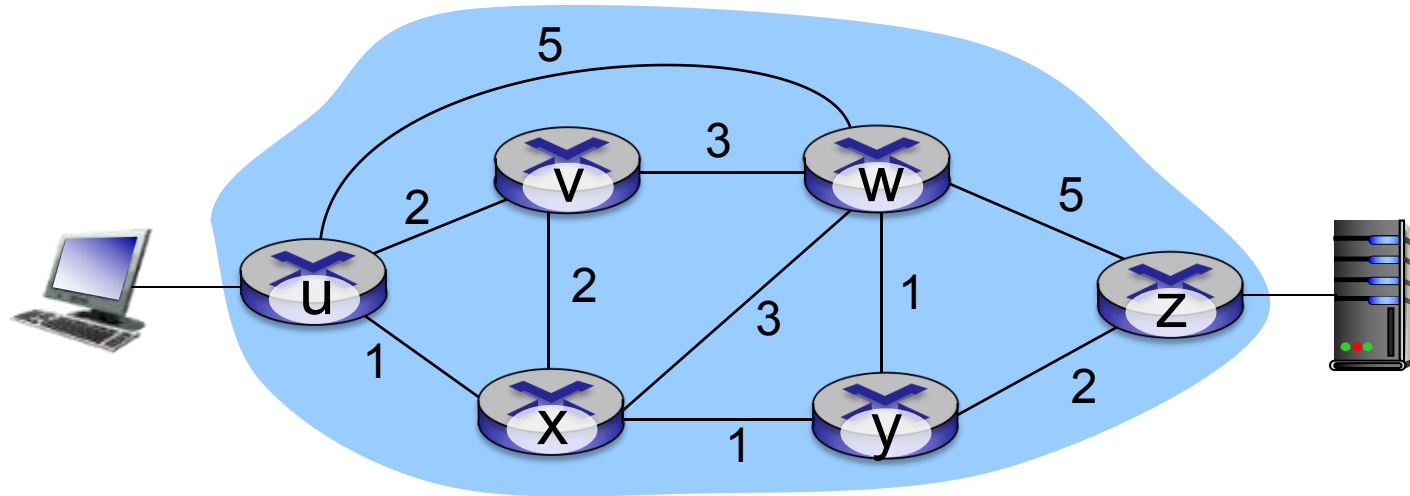
Vertically integrated
Closed, proprietary
Slow innovation
Small industry



Horizontal
Open interfaces
Rapid innovation
Huge industry



Traffic engineering: difficult traditional routing

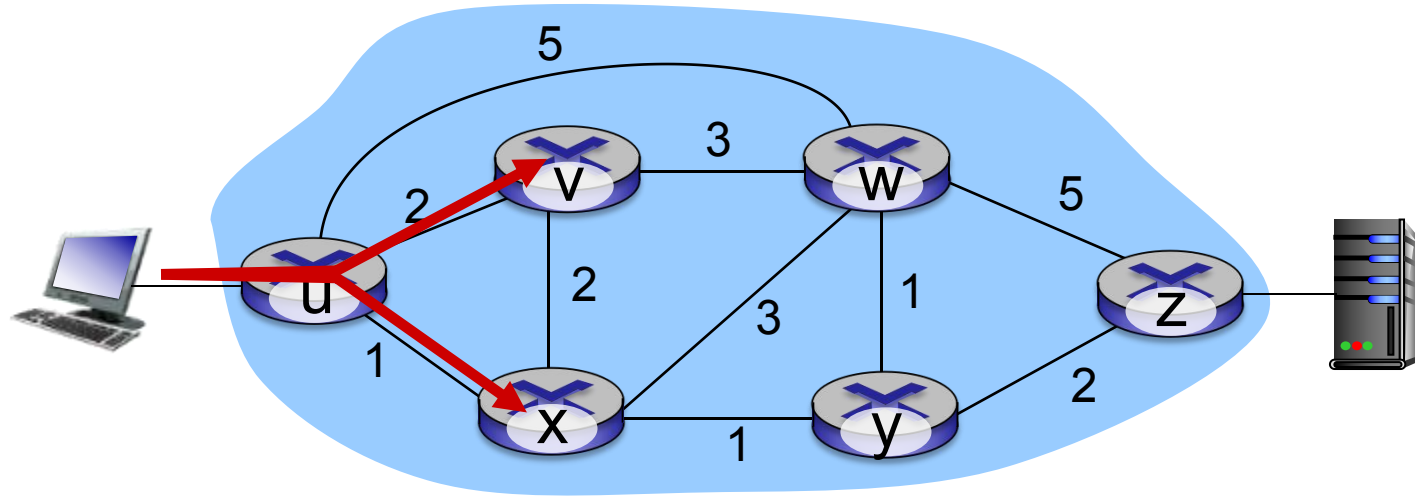


Q: what if network operator wants u-to-z traffic to flow along $uvwz$, x-to-z traffic to flow $xwyz$?

A: need to define **link weights** so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

But the link weights cannot be directly set to certain number

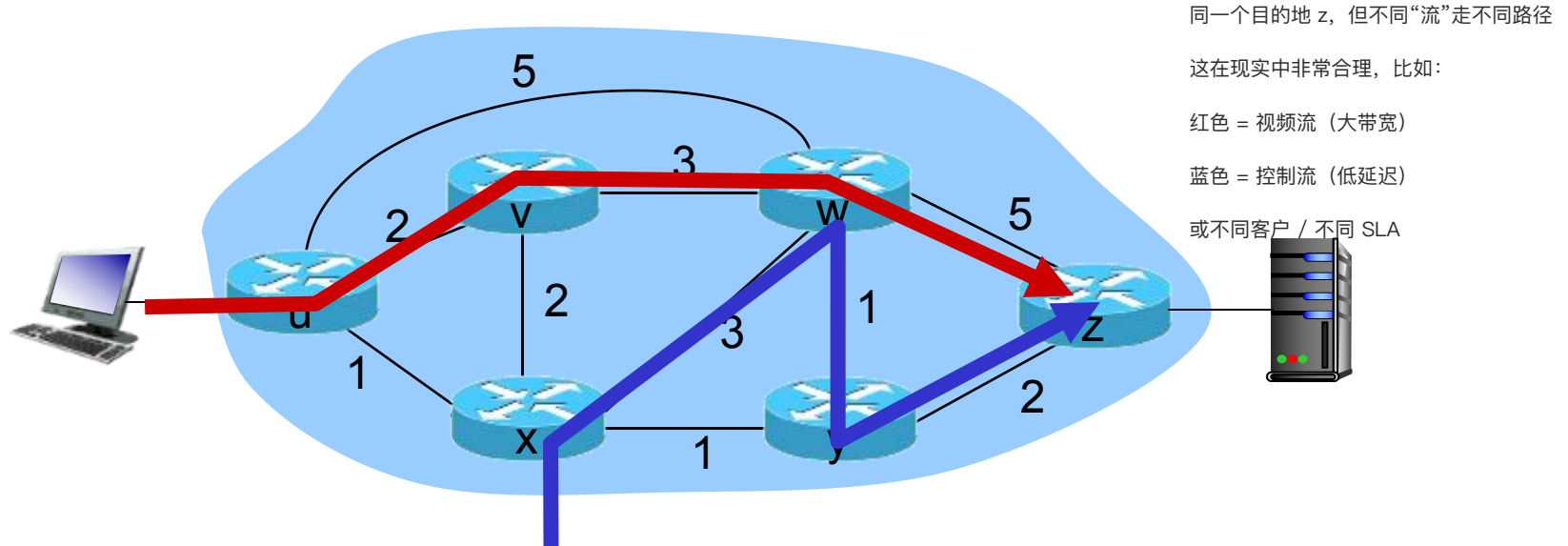
Traffic engineering: difficult



Q: what if network operator wants to split u-to-z traffic along uvwz *and* uxyz (load balancing)?

A: can't do it (or need a new routing algorithm)

Traffic engineering: difficult



Q: what if w wants to route blue and red traffic differently?

A: can't do it (with destination based forwarding, and LS, DV routing)

在传统 IP 路由（LS / DV）里，做“精细化流量工程”非常难，甚至做不到。

Software defined networking (SDN)

4. *programmable control applications*

routing

access control

load balance

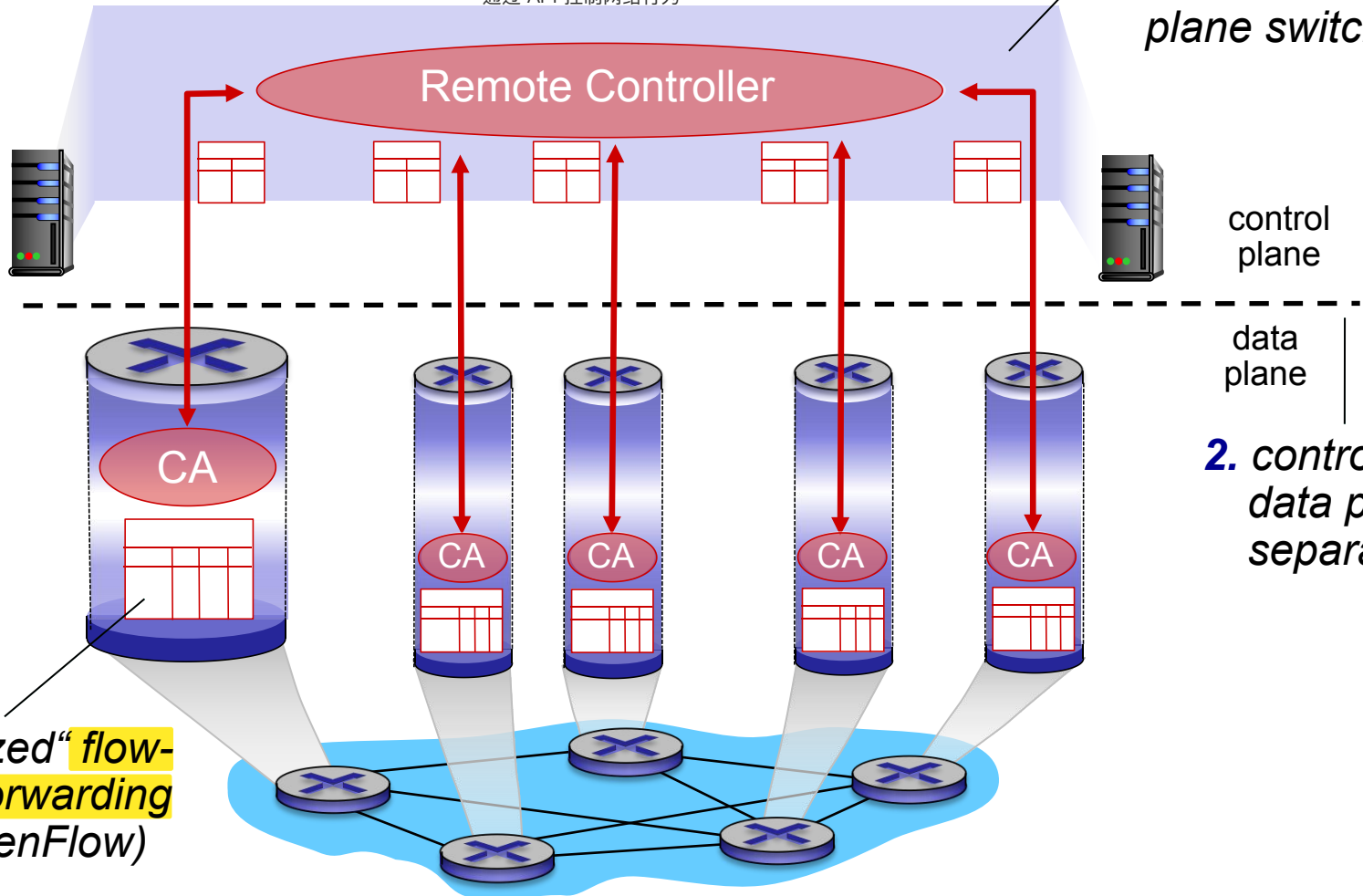
这些应用:

不是写在交换机里

是普通软件 (Java / Python / Go)

通过 API 控制网络行为

3. *control plane functions external to data-plane switches*



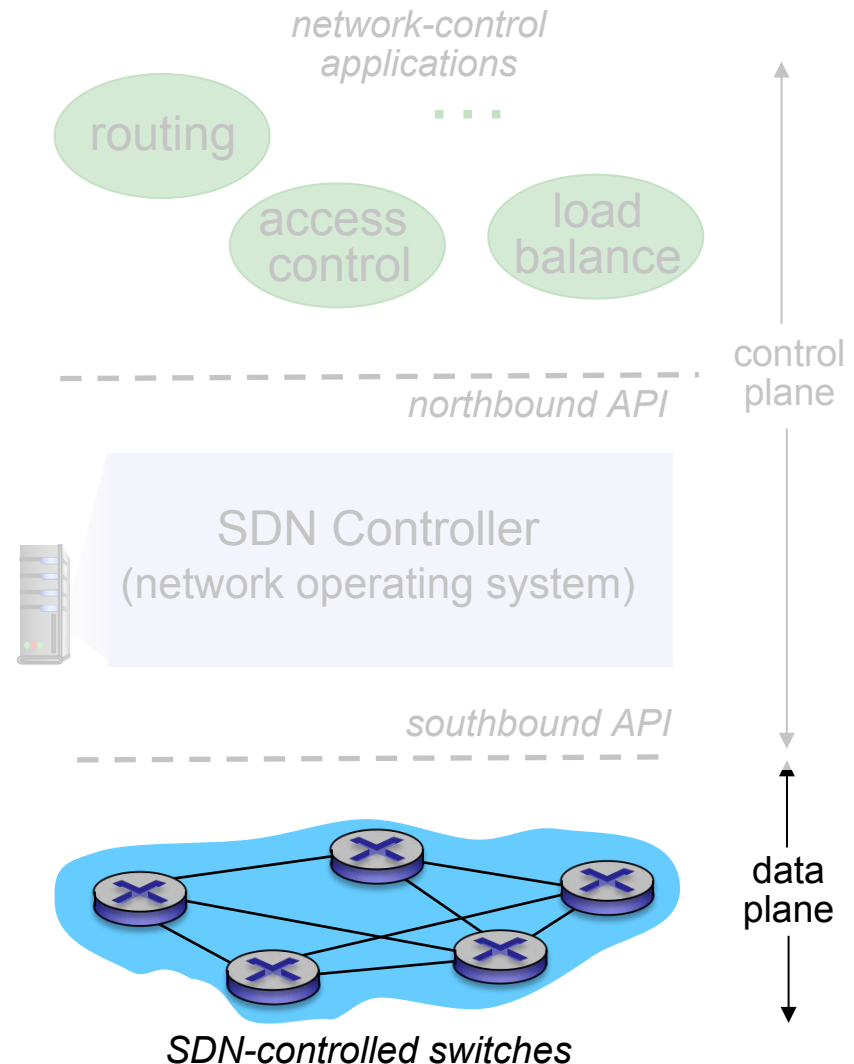
1. *generalized "flow-based" forwarding (e.g., OpenFlow)*

2. *control, data plane separation*

SDN perspective: data plane switches

Data plane switches

- fast, simple, commodity switches implementing generalized data-plane forwarding (Section 4.4) in hardware
- switch flow table computed, installed by controller
- API for table-based switch control (e.g., OpenFlow)
 - defines what is controllable and what is not
- protocol for communicating with controller (e.g., OpenFlow)

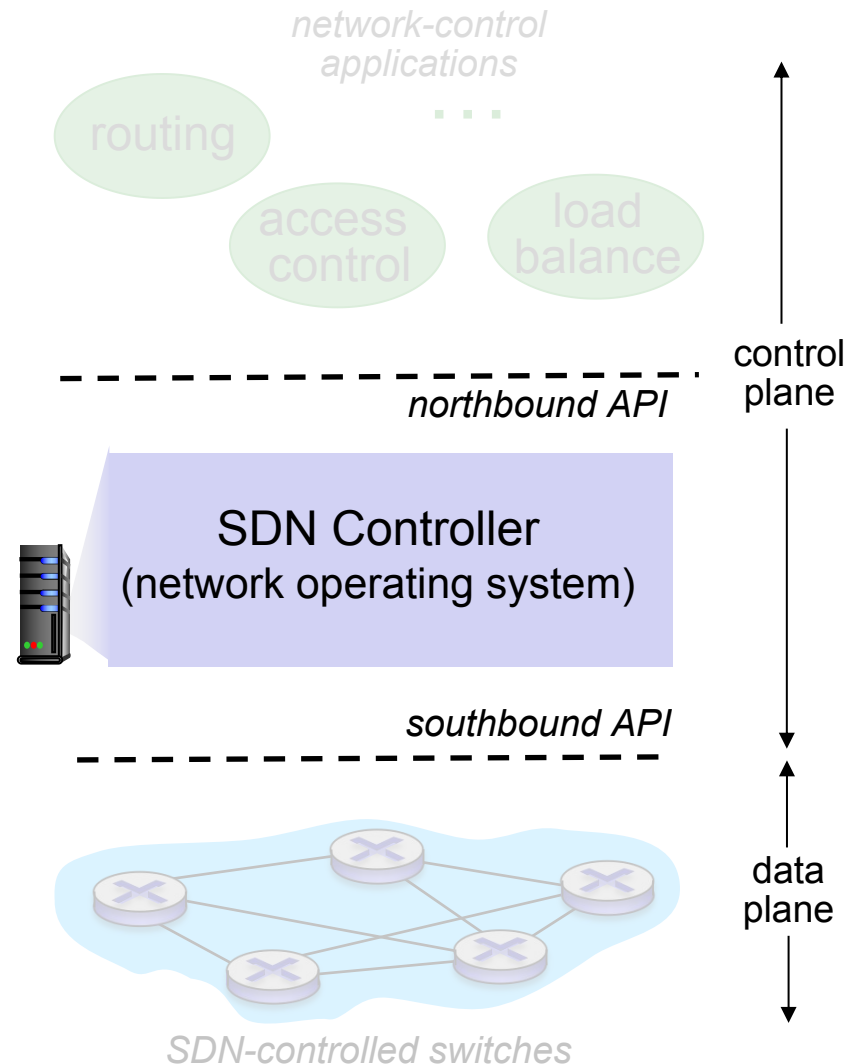


SDN perspective: SDN controller

SDN controller (network OS):

- maintain network state information
- interacts with network control applications “above” via northbound API
- interacts with network switches “below” via southbound API
- implemented as **distributed** system for performance, scalability, fault-tolerance, robustness

虽然是逻辑集中，但不是一台机器：
多控制器
主备
分片
容错
逻辑集中 ≠ 物理集中

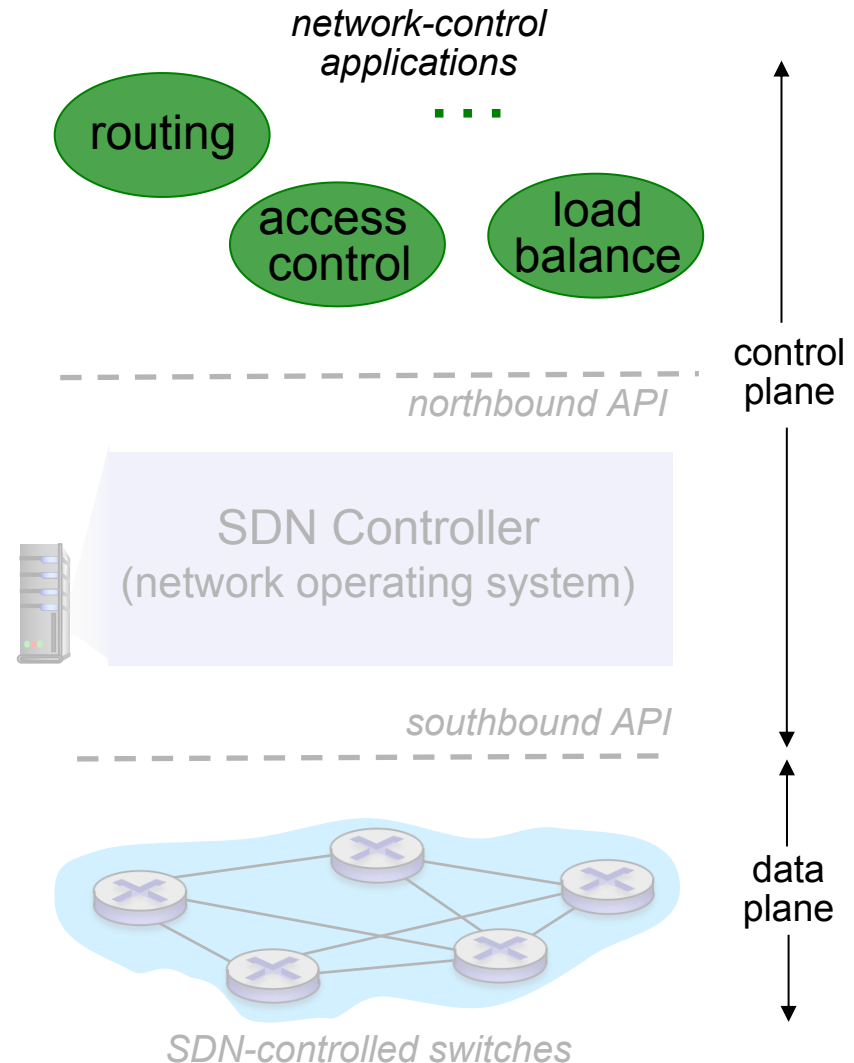


SDN perspective: control applications

network-control apps:

- “brains” of control: implement control functions using lower-level services, API provided by SDN controller
- *unbundled*: can be provided by 3rd party: distinct from routing vendor, or SDN controller

控制逻辑不再绑死在设备厂商里

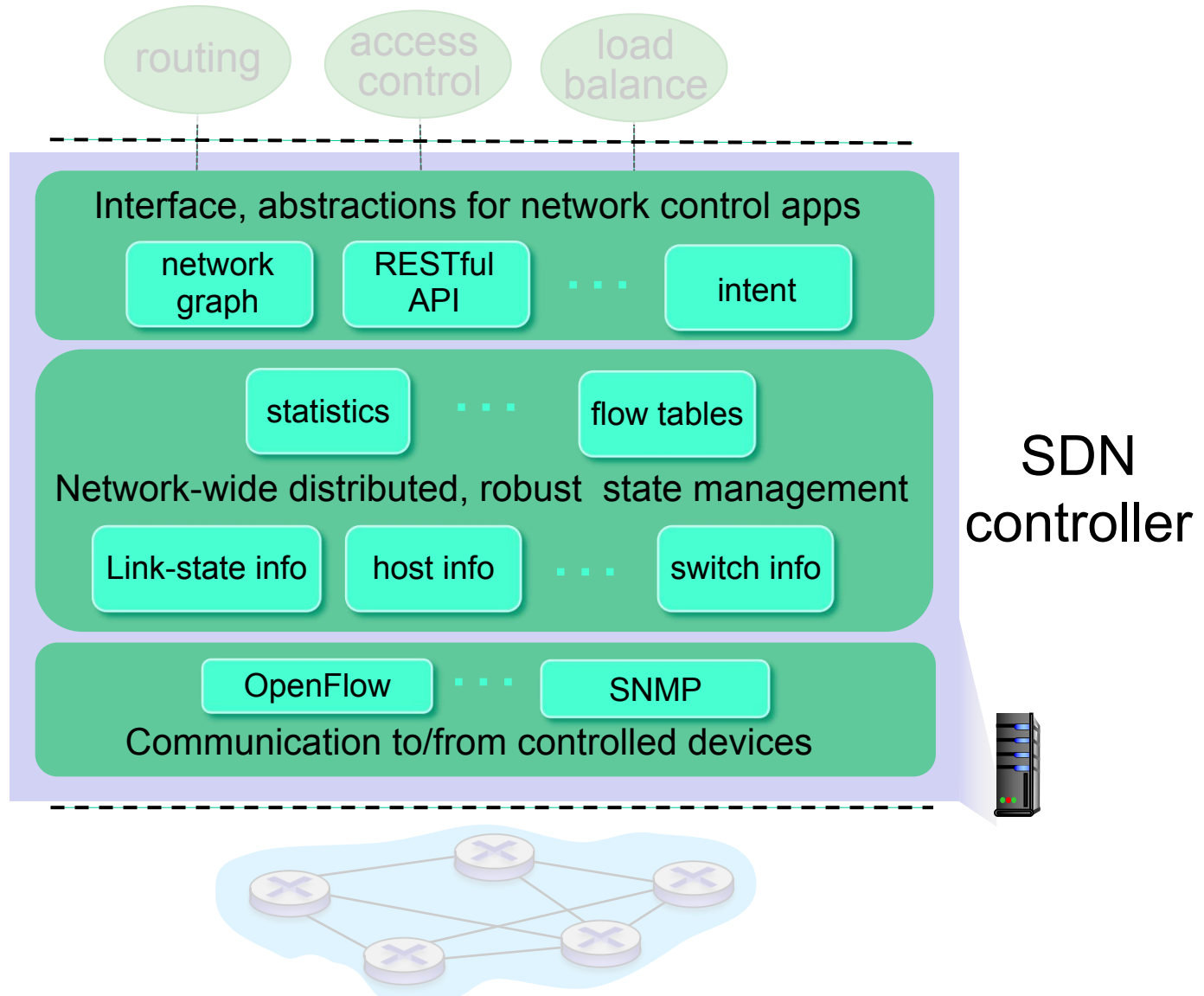


Components of SDN controller

Interface layer to
network control
apps: abstractions
API

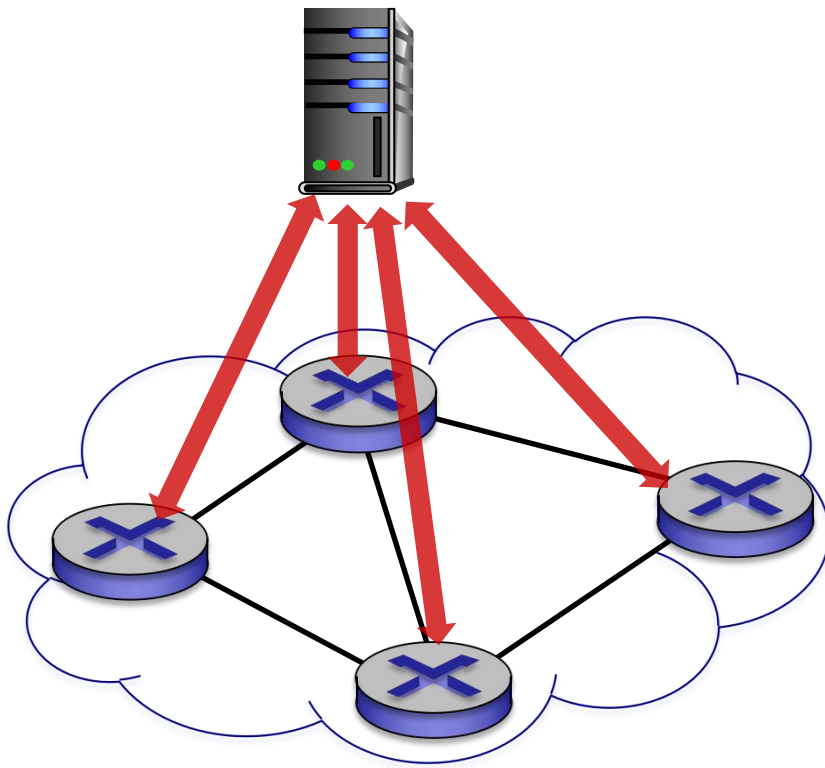
Network-wide state
management layer:
state of networks
links, switches,
services: a
distributed database

communication
layer:
communicate
between SDN
controller and
controlled switches



OpenFlow protocol

OpenFlow Controller



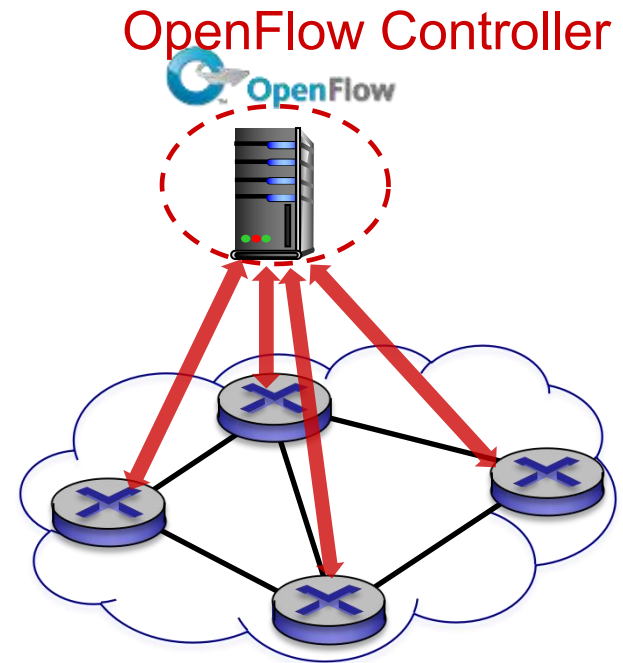
- operates between controller, switch
- TCP used to exchange messages
- OpenFlow messages:
 - controller-to-switch
 - switch to controller

OpenFlow: controller-to-switch messages

Key controller-to-switch messages

- *configure*: controller queries/sets switch configuration parameters
- *modify-state*: add, delete, modify flow entries in the OpenFlow tables
- *Read-state*: collect statistics and counter values from the switch's flow table and ports
- *packet-out*: controller can send this packet out of specific switch port

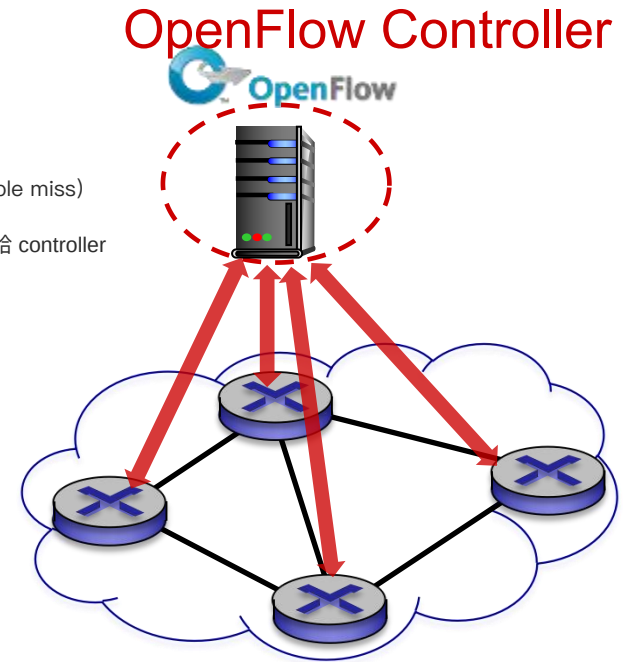
临时转发一个包



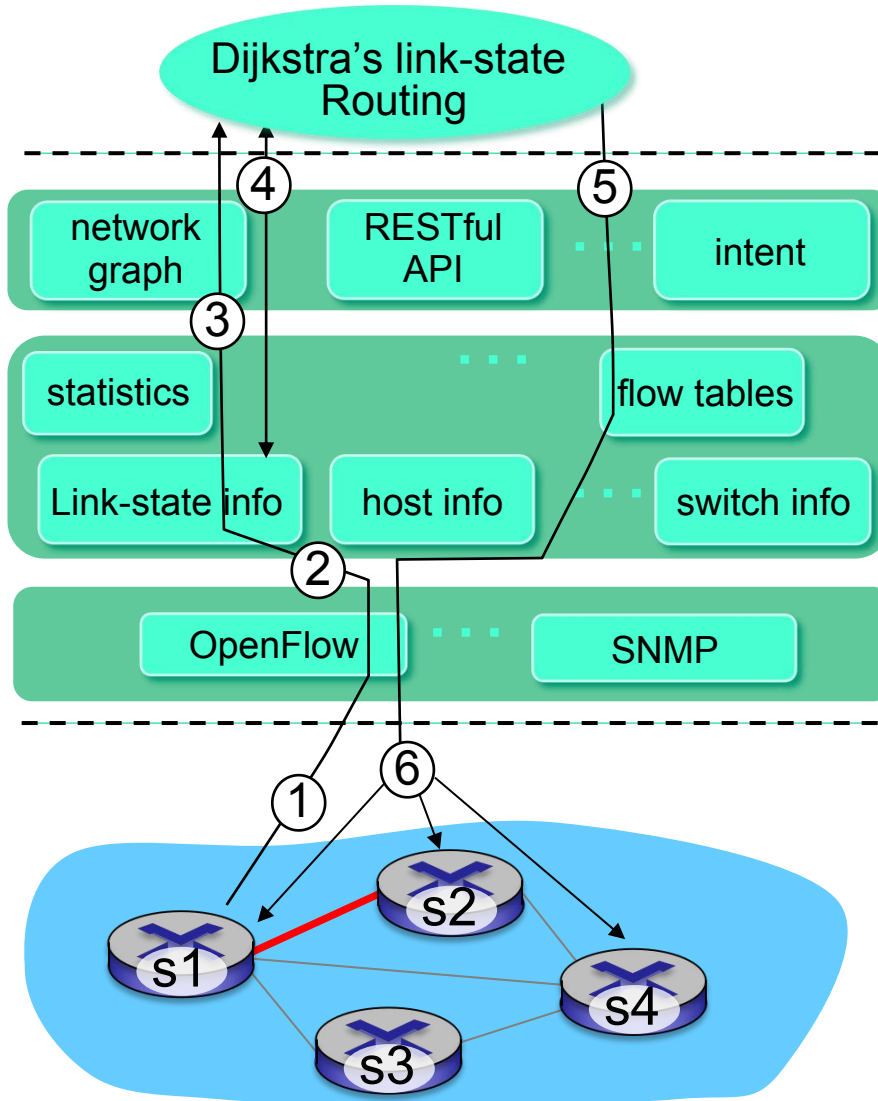
OpenFlow: switch-to-controller messages

Key switch-to-controller messages

- *packet-in*: transfer packet (and its control) to controller. See packet-out message from controller
匹配条件
没有匹配的 flow (table miss)
flow 规则要求把包送给 controller
- *flow-removed*: flow table entry deleted at switch
- *port status*: inform controller of a change on a port.

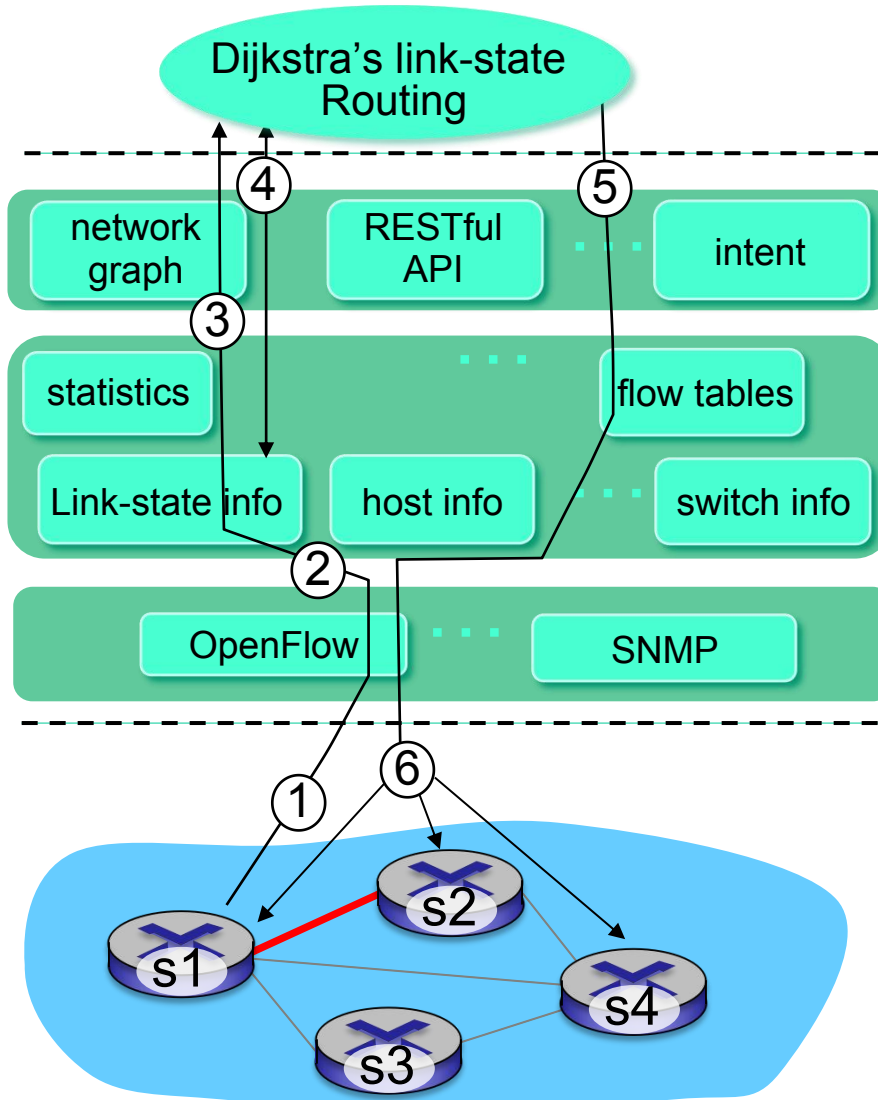


SDN: control/data plane interaction example



- ① S1, experiencing link failure using OpenFlow *port-status* message to notify controller
- ② SDN controller receives OpenFlow message, updates link status info
- ③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.
- ④ Dijkstra's routing algorithm access network graph info, link state info in controller, computes new routes

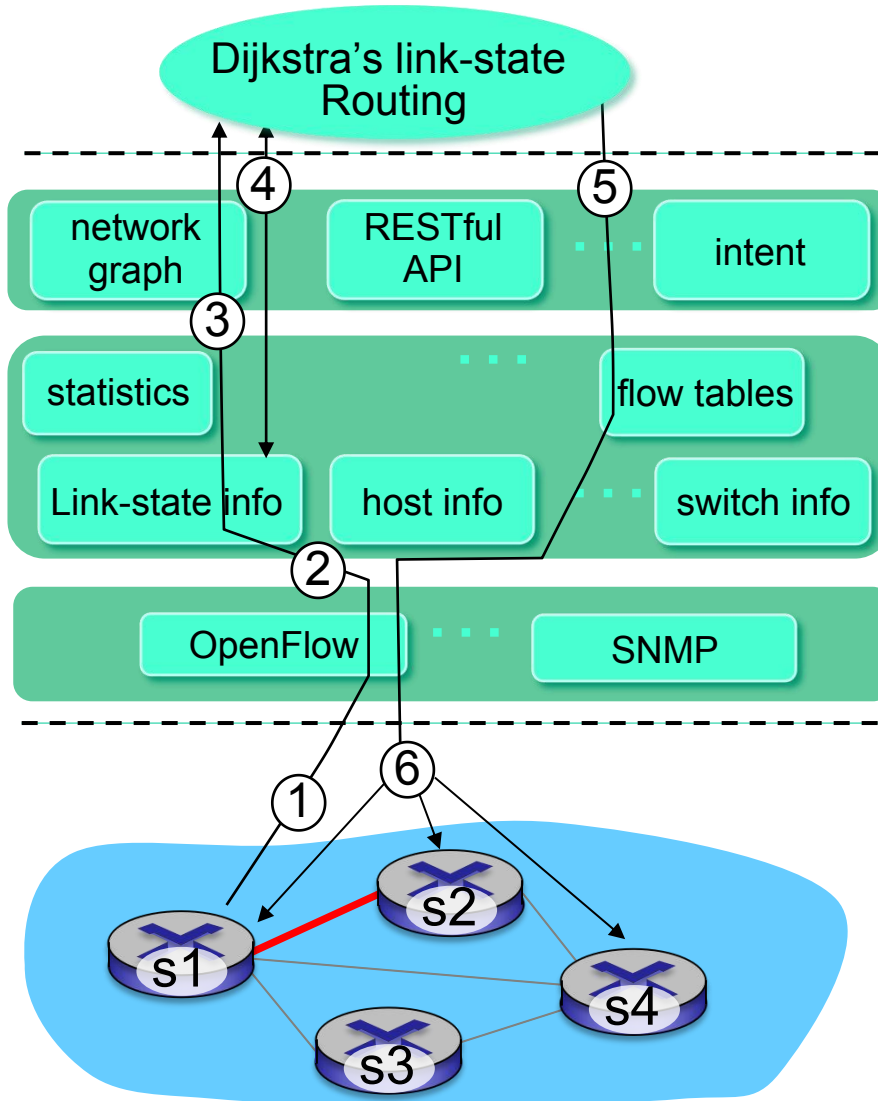
SDN: control/data plane interaction example



Two important differences from the earlier per-router-control scenario:

- Dijkstra's algorithm is executed as a **separate application**, outside of the packet switches.
- Packet switches send link updates to the **SDN controller** and not to each other.

SDN: control/data plane interaction example



- ⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed
- ⑥ Controller uses OpenFlow to install new tables in switches that need updating

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

ICMP: internet control message protocol

- used by hosts & routers to communicate network-level information

- error reporting: unreachable host, network, port, protocol
- echo request/reply (used by ping)

- network-layer “above” IP:

- ICMP msgs carried in IP datagrams

ICMP 不是 TCP / UDP 之上的协议

- ICMP message:

ICMP 直接封装在 IP 数据报里

- Type + code + the header and the first 8 bytes of IP datagram causing error

type+code+出错的 IP 包头 + 前 8 字节（方便源主机定位是哪个包出错）

<u>Type</u>	<u>Code</u>	<u>description</u>
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Traceroute and ICMP

- source sends series of UDP segments to destination

- first set has TTL = 1
- second set has TTL=2, etc.
- **unlikely port number**

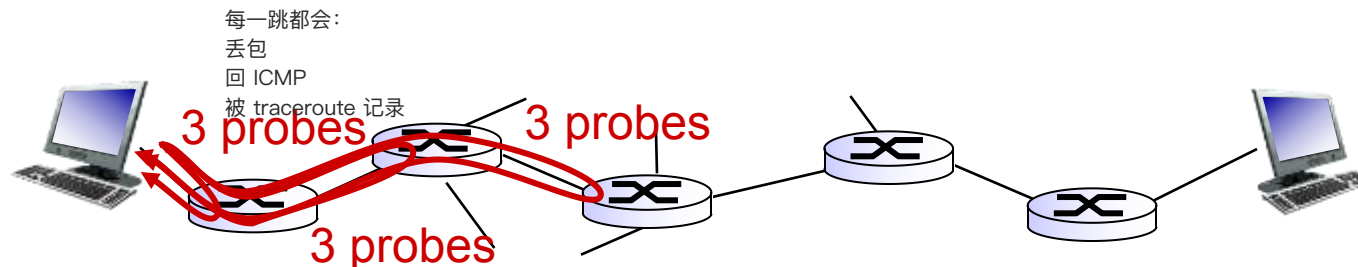
- when datagram in n th set arrives to n th router:

- router discards datagram and sends source ICMP message (type 11, code 0)
- ICMP message include name of router & IP address

- when ICMP message arrives, source records RTTs

stopping criteria: traceroute 发送 UDP 到一个几乎不可能开放的端口
目的主机最终收到 UDP 包后:
发现端口没人监听
返回 ICMP Port Unreachable (Type 3, Code 3)
这就是 traceroute 的 停止条件

- UDP segment eventually arrives at destination host
- destination **returns ICMP “port unreachable” message (type 3, code 3)**
- source stops



Chapter 5: summary

we've learned a lot!

- approaches to network control plane
 - per-router control (traditional)
 - logically centralized control (software defined networking)
- traditional routing algorithms
 - implementation in Internet: OSPF, BGP
- SDN controllers
 - implementation in practice: ODL, ONOS
- Internet Control Message Protocol
- (network management)

next stop: link layer!

Chapter 6: Link layer and LANs

our goals:

- understand principles behind link layer services:
 - error detection, correction
 - sharing a broadcast channel: multiple access
 - link layer addressing
 - local area networks: Ethernet, VLANs
- instantiation, implementation of various link layer technologies

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

6.5 link virtualization: MPLS

6.6 data center networking

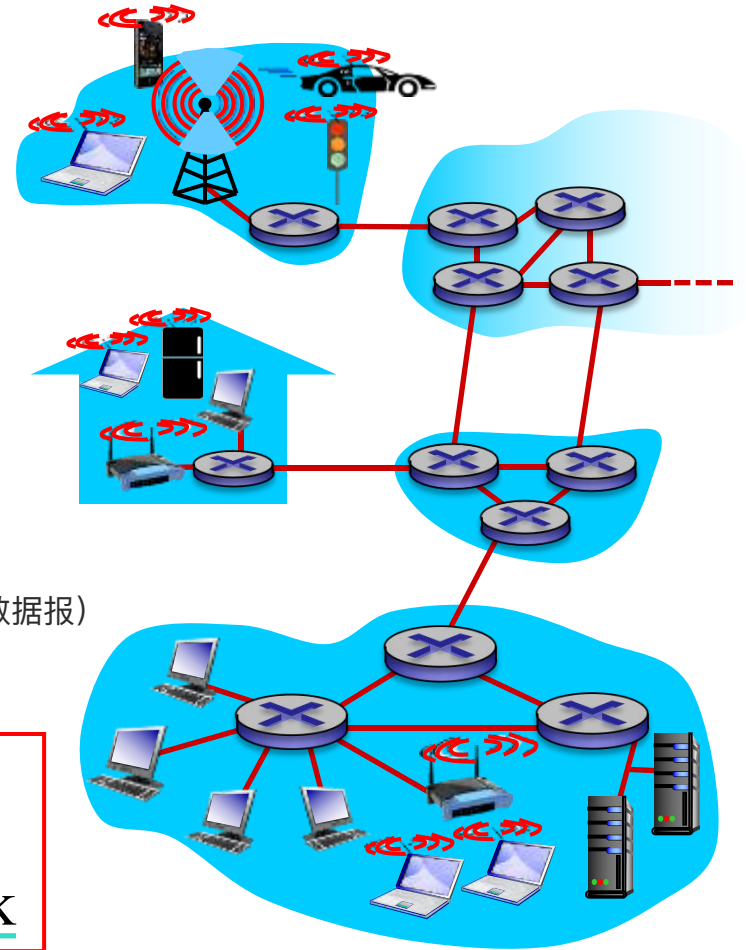
6.7 a day in the life of a web request

Link layer: introduction

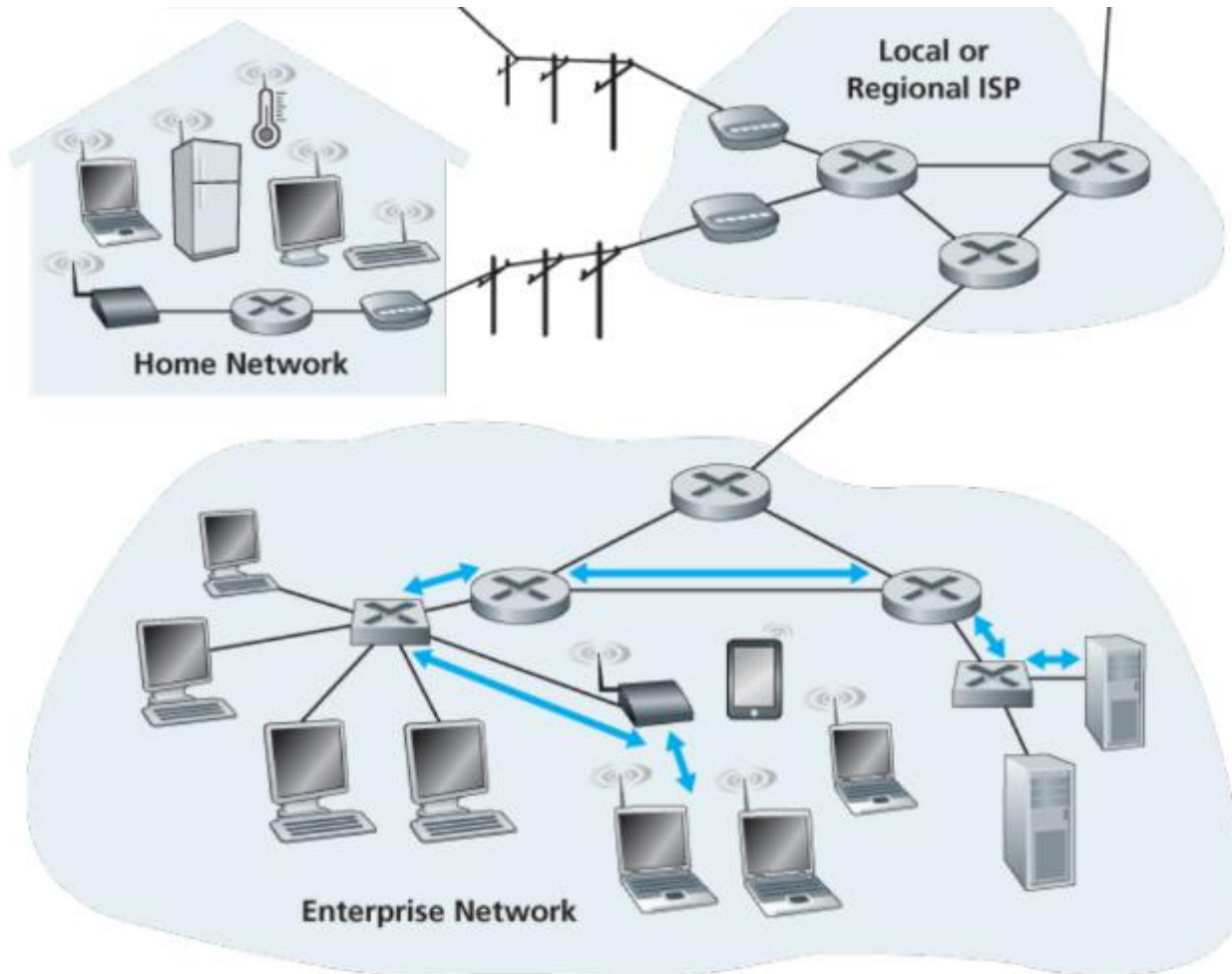
terminology:

- hosts and routers: **nodes**
 - communication channels that connect adjacent nodes along communication path: **links**
 - wired links
 - wireless links
 - layer-2 packet: **frame**,
encapsulates datagram
- 网络层: datagram (IP 数据报)
• 链路层: frame (帧)

link layer has responsibility of transferring datagram from one node to *physically adjacent* node over a link



Link layer: introduction



Link layer: context

IP 数据报 (datagram) 是端到端的

- datagram transferred by different link protocols over different links:
 - e.g., Ethernet on first link, PPP on intermediate links, 802.11 on last link
- each link protocol provides different services
 - e.g., may or may not provide rdt over link

但在路上:

每一跳 (link)

都可能使用不同的链路层协议

transportation analogy:

- trip from SUSTech to Tsinghua
 - metro: SUSTech to SZ North
 - High speed train: SZ North to Beijing West
 - taxi: Beijing West to Tsinghua
- tourist = **datagram**
- transport segment = **communication link** 链路
- transportation mode = **link layer protocol**
- travel agent = **routing algorithm**

Link layer services

■ *framing, link access:*

- encapsulate datagram into frame, adding header, trailer
- channel access if shared medium
- “MAC” addresses used in frame headers to identify source, destination
 - different from IP address!

mac是物理地址

■ *reliable delivery between adjacent nodes*

- we learned how to do this already (chapter 3)!
- seldom used on low bit-error link (fiber, some twisted pair)
- wireless links: high error rates

相邻节点之间的可靠传输 (Reliable delivery)

只在“一跳”内

常见于:

无线链路 (误码率高)

不常见于:

光纤、有线 (误码率很低)

- *Q*: why both link-level and end-end reliability?

为什么既要链路层可靠性, 又要端到端 (TCP) 可靠性?

链路层:

局部快速重传, 减少无线错误影响

T

CP:

端到端兜底, 防止中间节点丢包/乱序

Link layer services (more)

■ *flow control:*

- pacing between adjacent sending and receiving nodes

■ *error detection:*

- errors caused by signal attenuation, noise.
信号衰减和噪声
- receiver **detects presence of errors:**
 - signals sender for retransmission or drops frame

■ *error correction:*

- receiver identifies *and corrects* bit error(s) without resorting to retransmission
典型于
• 无线链路
• FEC (前向纠错)

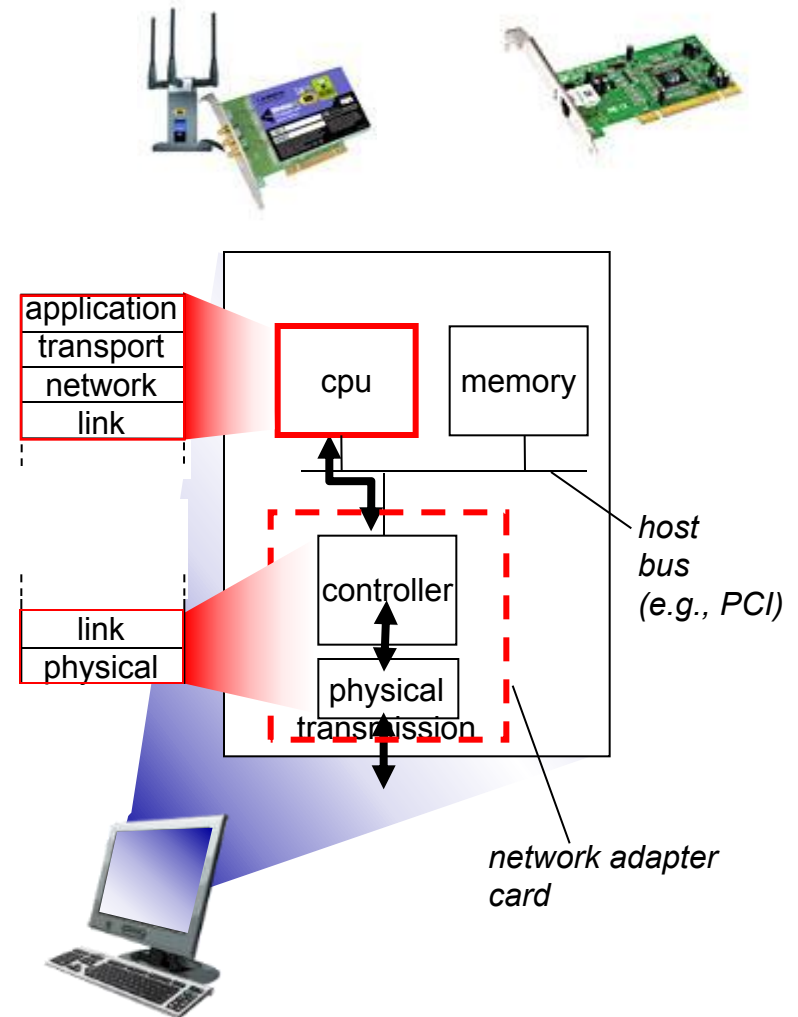
■ *half-duplex and full-duplex*

- with half duplex, nodes at both ends of link can transmit, but not at same time
半双工:
• 双方都能发
• 不能同时发 (如老式以太网)

全双工:

Where is the link layer implemented?

- in each and every host
- link layer implemented in “adaptor” (aka network interface card 网络适配器 NIC) or on a chip
 - Ethernet card, 802.11 card; Ethernet chipset
 - implements link, physical layer
- attaches into host's system buses
- combination of hardware, software, firmware



Adaptors communicating

发送端

1 从网络层接收 datagram

2 封装成 frame

3 加入：

- MAC 地址
- CRC
- 流控 / 重传机制

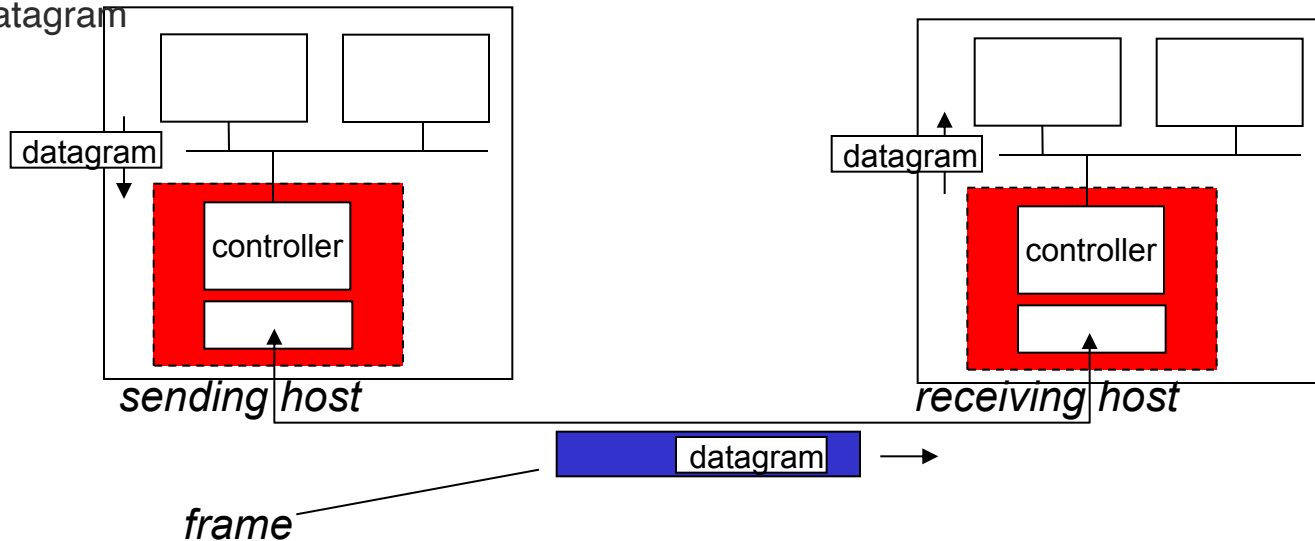
4 发到链路上

接收端

1 接收 frame

2 检查错误

3 去掉帧头帧尾



■ sending side:

- encapsulates datagram in frame
- adds error checking bits, rdt, flow control, etc.

■ receiving side

- looks for errors, rdt, flow control, etc.
- extracts datagram, passes to upper layer at receiving side

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

6.5 link virtualization: MPLS

6.6 data center networking

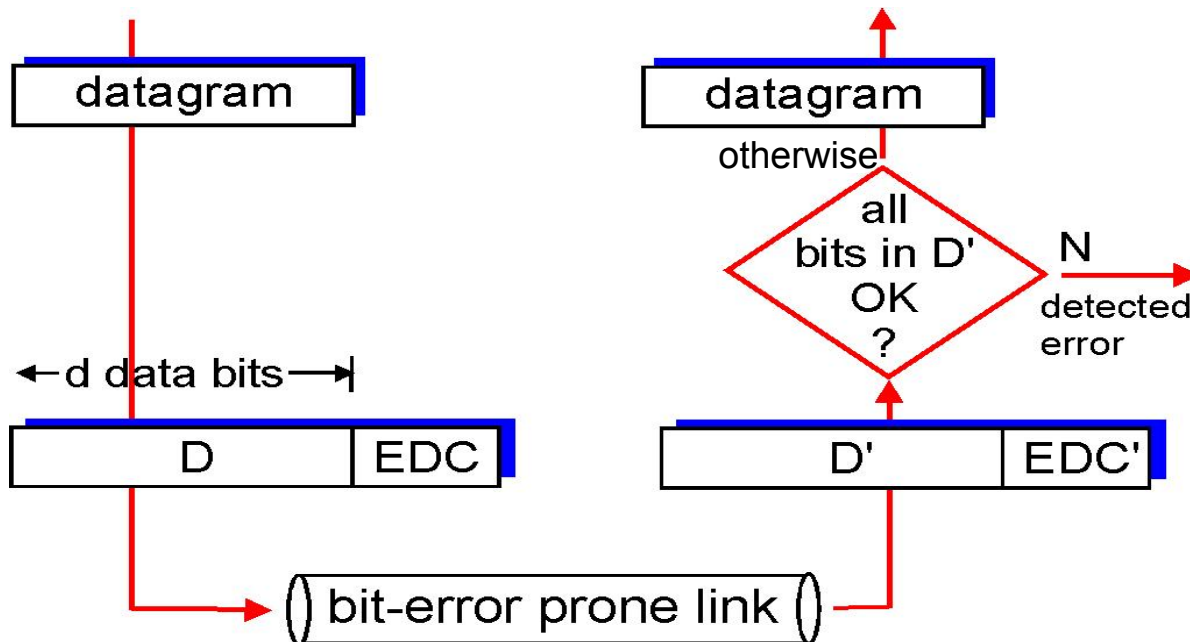
6.7 a day in the life of a web request

Error detection

EDC= Error Detection and Correction bits

D = Data protected by error checking, may include header fields

- Error detection **not 100% reliable!**
 - protocol may miss some errors, but rarely
 - larger EDC field yields better detection and correction, but larger overhead

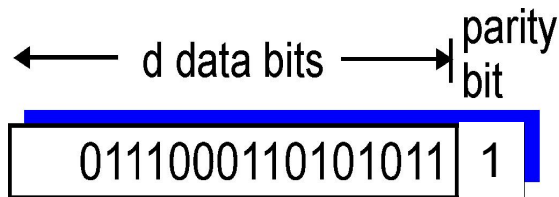


- Parity checks
- Check-summing methods
- Cyclic-redundancy check

Parity checking

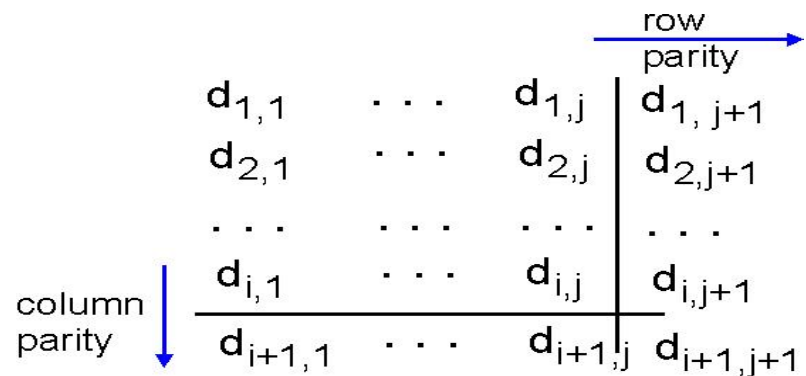
single bit parity:

- detect single bit errors
- Even parity scheme
- Odd parity scheme



two-dimensional bit parity:

- detect and correct single bit errors



1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

no errors

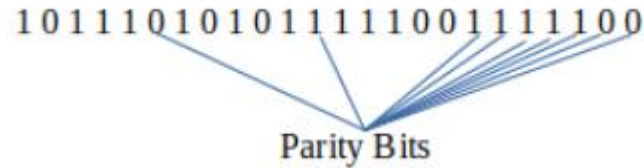
1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

parity error

*correctable
single bit error*

Parity checking

1	0	1	1	1	0
1	0	1	0	1	1
1	1	1	0	0	1
1	1	1	1	0	0



Case 1: a bit is in error.

1	0	1	1	1	0
1	0	0	0	1	1
1	1	1	0	0	1
1	1	1	1	0	0

Error Detected

Case 2: two bits are in error.

0	0	1	1	1	0
1	0	1	0	1	1
1	1	1	0	1	1
1	1	1	1	0	0

Correct Bit Detect As Incorrect Bit

Error Detected

Case 3: error not detected

1	0	1	1	1	0
1	0	0	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

Not Detected so not Corrected

Many other cases ...

Internet checksum (review)

goal: detect “errors” (e.g., flipped bits) in transmitted packet
(note: used at transport layer only)

sender:

- treat segment contents as sequence of 16-bit integers
- checksum: addition (1's complement sum) of segment contents
- sender puts checksum value into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
 - NO - error detected
 - YES - no error detected.
But maybe errors nonetheless?

Cyclic redundancy check

- more powerful error-detection coding
- view data bits, **D**, as a binary number
- choose $r+1$ bit pattern (generator), **G**
- goal: choose r CRC bits, **R**, such that
 - $\langle D, R \rangle$ exactly divisible by G (modulo 2)
 - receiver knows G , divides $\langle D, R \rangle$ by G . If non-zero remainder: error detected!
 - can detect all consecutive bit errors of r bits or less
- widely used in practice (Ethernet, 802.11 WiFi, ATM)

$$1011 \text{ XOR } 0101 = 1110$$

$$1001 \text{ XOR } 1101 = 0100$$

$$1011 - 0101 = 1110$$

$$1001 - 1101 = 0100$$

选择 R , 使得拼接后的比特串:

$\langle D, R \rangle = D$ 后面加上 R

能被 G 做 “模2除法” 整除 (余数为 0)。

← d bits → ← r bits →



*bit
pattern*

$$D * 2^r \text{ XOR } R$$

*mathematical
formula*

Cyclic redundancy check

All CRC calculations are done in **modulo-2 arithmetic** without carries in addition or borrows in subtraction.

- This means that addition and subtraction are identical, and
- both are equivalent to the bitwise exclusive-or (XOR) of the operands.

$$1011 \text{ XOR } 0101 = 1110$$

$$1001 \text{ XOR } 1101 = 0100$$

$$1011 - 0101 = 1110$$

$$1001 - 1101 = 0100$$

Multiplication and division are the same as in base-2 arithmetic, except that any required addition or subtraction is done **without carries or borrows**.

任何长度 $\ll R$ 的连续突发错误一定能被检测出

$$\begin{array}{r} 10001 \text{ remainder } 101 \\ 10011 \overline{) 100100110} \\ \underline{10011} \\ 10110 \\ \underline{10011} \\ 101 \end{array}$$

CRC example

want: 左移 r 位

$$D \cdot 2^r \text{ XOR } R = nG$$

能被 G 整

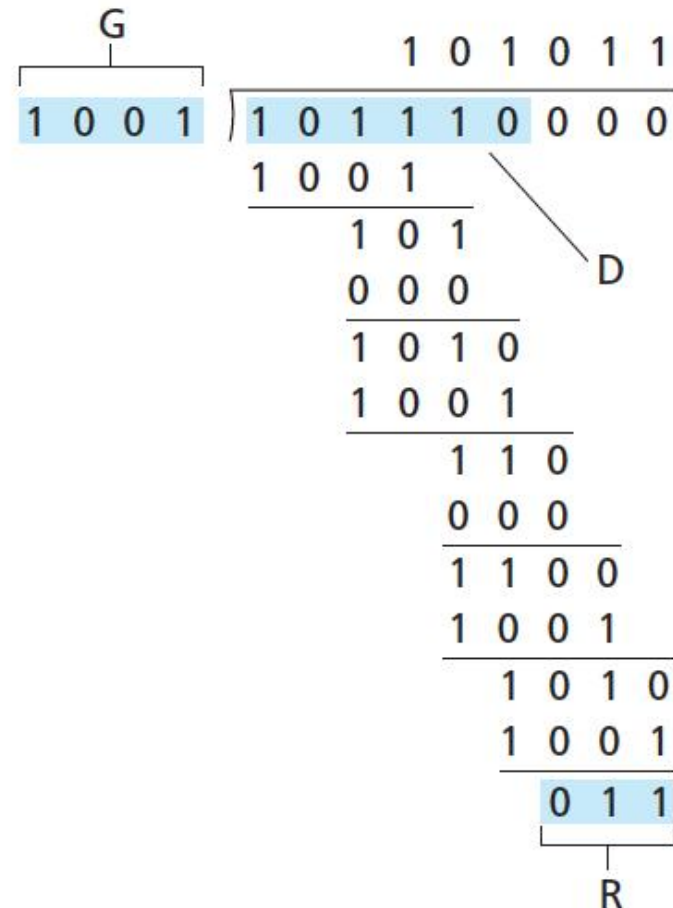
equivalently:

$$D \cdot 2^r = nG \text{ XOR } R$$

equivalently:

if we divide $D \cdot 2^r$ by G , want remainder R to satisfy:

$$R = \text{remainder}\left[\frac{D \cdot 2^r}{G}\right]$$



Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs

- addressing, ARP
- Ethernet
- switches
- VLANs

6.5 link virtualization: MPLS

6.6 data center networking

6.7 a day in the life of a web request

Multiple access links, protocols

当一条链路被多个节点共享时，这种链路叫 multiple access link（多路访问链路）

two types of “links”:

- point-to-point

- PPP for dial-up access
- point-to-point link between Ethernet switch, host

只有两个人用
不需要抢
不存在冲突 (collision)
谁想发就能发

- *broadcast (shared wire or medium)*

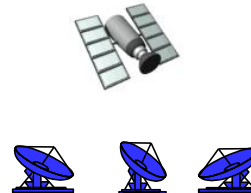
- old-fashioned Ethernet
- 802.11 wireless LAN



shared wire (e.g.,
cabled Ethernet)



shared RF
(e.g., 802.11 WiFi)



shared RF
(satellite)



humans at a
cocktail party
(shared air, acoustical)

Multiple access protocols

- single shared broadcast channel
- two or more simultaneous transmissions by nodes: interference
 - **collision** if node receives two or more signals at the same time

一个节点在同一时间接收到两个或以上信号

multiple access protocol

- **distributed algorithm** that determines how nodes share channel, i.e., determine which and when node can transmit
- communication about channel sharing **must use channel itself!**
 - no out-of-band channel for coordination

没有带外信道

An ideal multiple access protocol

given: broadcast channel of rate R bps

Desired properties:

1. when one node wants to transmit, it can send at rate R .
2. when M nodes want to transmit, each can send at average rate R/M
3. **fully decentralized:**
 - no special node to coordinate transmissions
 - no synchronization of clocks, slots 每个node本地做决定
4. simple

MAC protocols: taxonomy

three broad classes:

- *channel partitioning*

- divide channel into smaller “pieces” (time slots, frequency, code) 每个节点 只用自己的那一块
- allocate piece to node for exclusive use 每个节点只用自己的那一块，如果节点没数据就会浪费

- *random access*

- channel not divided, allow collisions
- “recover” from collisions 冲突多的时候浪费带宽

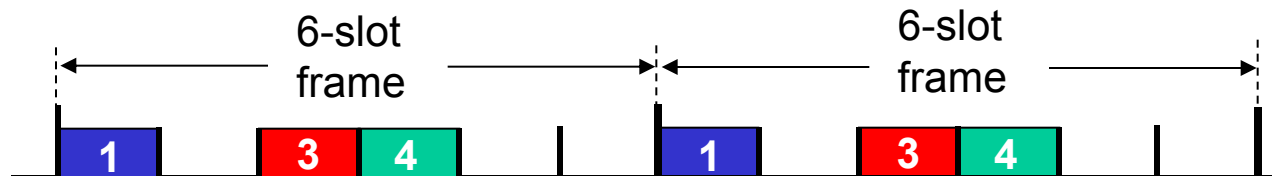
- *“taking turns”*

- nodes take turns, but nodes with more to send can take longer turns

Channel partitioning MAC protocols: TDMA

TDMA: time division multiple access

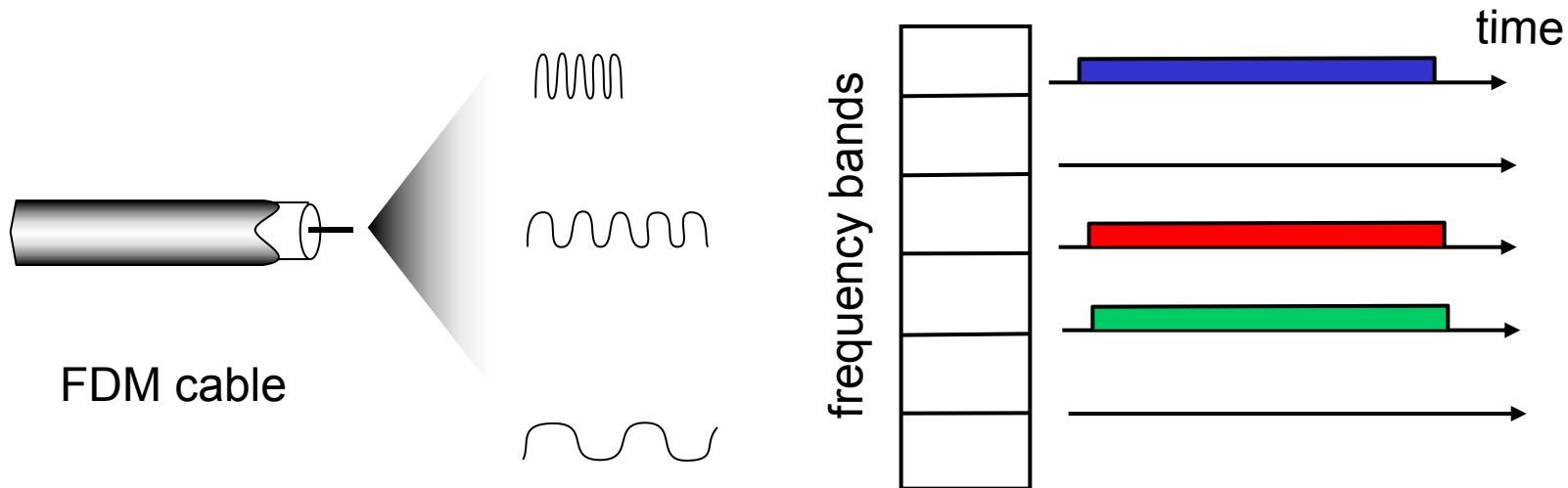
- access to channel in "rounds"
- each station gets fixed length slot (length = packet transmission time) in each round
- unused slots go idle
- example: 6-station LAN, 1,3,4 have packets to send, slots 2,5,6 idle



Channel partitioning MAC protocols: FDMA

FDMA: frequency division multiple access

- channel spectrum divided into frequency bands
- each station assigned fixed frequency band
- unused transmission time in frequency bands go idle
- example: 6-station LAN, 1,3,4 have packet to send, frequency bands 2,5,6 idle



Channel partitioning: limitations

Desired properties:

1. **【x】** when one node wants to transmit, it can send at rate R .
2. **【√】** when M nodes want to transmit, each can send at average rate R/M
3. fully decentralized:
 - no special node to coordinate transmissions
 - no synchronization of clocks, slots
4. simple

Random access protocols

- Random access; if fails, wait for a random time
- two or more transmitting nodes → “collision”,
- random access MAC protocol specifies:
 - how to **detect** collisions
 - how to **recover** from collisions (e.g., via delayed retransmissions)
- **Advantage:** when node has packet to send
 - transmit at full channel data rate R .
 - no *a priori* coordination among nodes 不需要事先协调/排时间表/同步时钟
- examples of random access MAC protocols:
 - slotted ALOHA
 - ALOHA
 - CSMA, CSMA/CD (Ethernet), CSMA/CA (802.11)

Slotted ALOHA

assumptions:

- all frames same size
- time divided into equal size slots (time to transmit 1 frame)
- nodes **start to transmit only slot beginning**
- nodes are synchronized
- if 2 or more nodes transmit in slot, all nodes detect collision

operation:

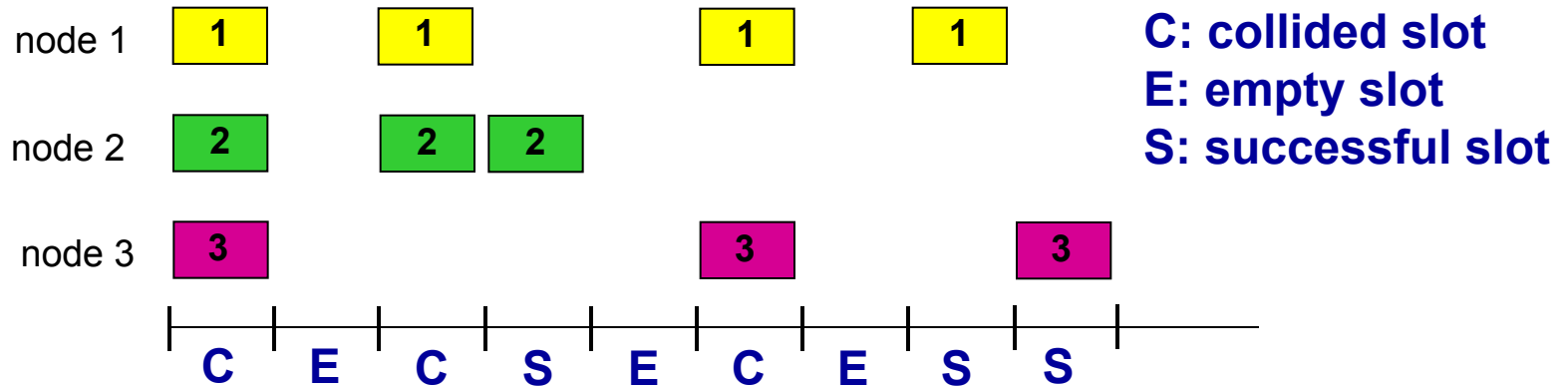
节点一旦有了新帧 (fresh frame)，不会立刻发，而是等到下一个时隙起点再发。

when node obtains fresh frame,
transmits in next slot

- *if no collision:* node can send new frame in next slot
- *if collision:* node retransmits frame in each subsequent slot with probability p until success

如果发生冲突，节点不会每个后续时隙都“必发”（那样大家永远撞）。而早在每个后续时隙中以概率 p 决定是否重传，直到成功为止。

Slotted ALOHA



Pros:

- single active node can continuously transmit at full rate of channel
- highly decentralized: only slots in nodes need to be in sync 只要大家的时隙同步就行。
- simple

Cons:

- collisions, wasting slots
- idle slots
- clock synchronization

Slotted ALOHA: efficiency

efficiency: long-run fraction of successful slots
(many nodes, all with many frames to send)

- *suppose*: N nodes with many frames to send, each transmits in slot with probability p
- prob that given node has success in a slot $= p(1-p)^{N-1}$
- prob that *any* node has a success $= Np(1-p)^{N-1}$

- max efficiency: find p^* that maximizes $Np(1-p)^{N-1}$
- for many nodes, take limit of $Np^*(1-p^*)^{N-1}$ as N goes to infinity, gives: $p^*=1/N$

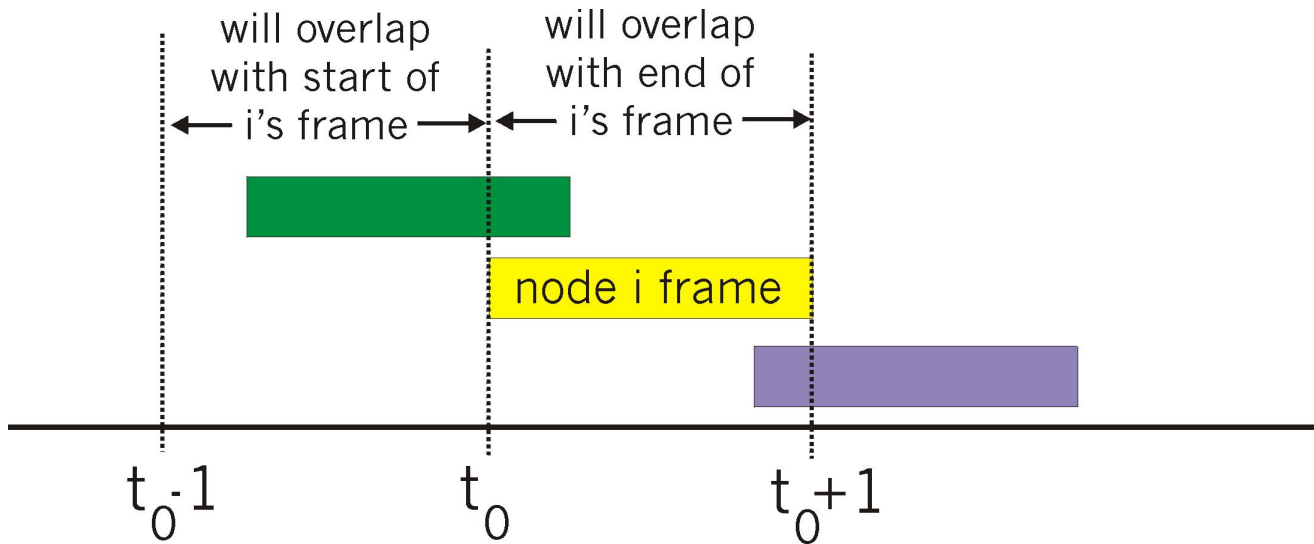
max efficiency $= 1/e = 0.37$

at best: channel used for useful transmissions 37% of time!



Pure (unslotted) ALOHA

- unslotted Aloha: simpler, no synchronization
- when frame first arrives
 - transmit immediately
- collision probability increases:
 - frame sent at t_0 collides with other frames sent in $[t_0-1, t_0+1]$



Pure ALOHA efficiency

$P(\text{success by given node}) = P(\text{node transmits}) \cdot$

$P(\text{no other node transmits in } [t_0-1, t_0]) \cdot$

$P(\text{no other node transmits in } [t_0, t_0+1])$

$$= p \cdot (1-p)^{N-1} \cdot (1-p)^{N-1}$$

$$= p \cdot (1-p)^{2(N-1)}$$

... choosing optimum p and then letting $n \rightarrow \infty$

$$= 1/(2e) = 0.18$$

even *worse* than slotted Aloha!

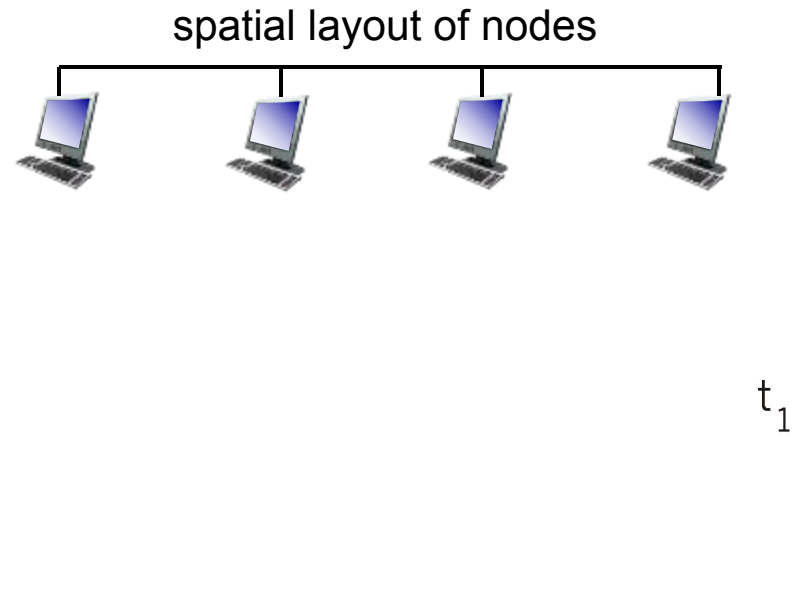
CSMA (carrier sense multiple access)

CSMA: listen before transmit:

- if channel sensed idle: transmit entire frame
 - if channel sensed busy, defer transmission
-
- human analogy: don't interrupt others!

CSMA collisions

- collisions *can* still occur: propagation delay means two nodes may not hear each other's transmission
- **collision**: entire packet transmission time wasted
 - distance & propagation delay play role in determining collision probability



信号不是瞬间到达所有节点的，它在电缆/无线中传播需要时间。

所以会出现这种情况：

- A 节点开始发了
- 但 A 的信号还没传播到 B 那边
- B 此时侦听到的仍然是“空闲”
- B 也开始发

CSMA/CD (collision detection)

CSMA/CD: CSMA+CD (collision detection)

- collision detection → stop talking: CSMA: 发送前先侦听信道 (空闲才发, 忙就等)

- collisions detected within short time
- colliding transmissions are aborted, reducing channel wastage 冲突没法完全避免, 但可以“尽快发现 + 尽快止损”。

■ Suitable scenarios:

- easy in **wired LANs**: measure signal strengths, compare transmitted, received signals
- difficult in wireless LANs: received signal strength overwhelmed by local transmission strength

- human analogy: the polite conversationalist

你在发送时, 可以测线上的信号

把“自己发出去的”与“线上实际看到的”对比

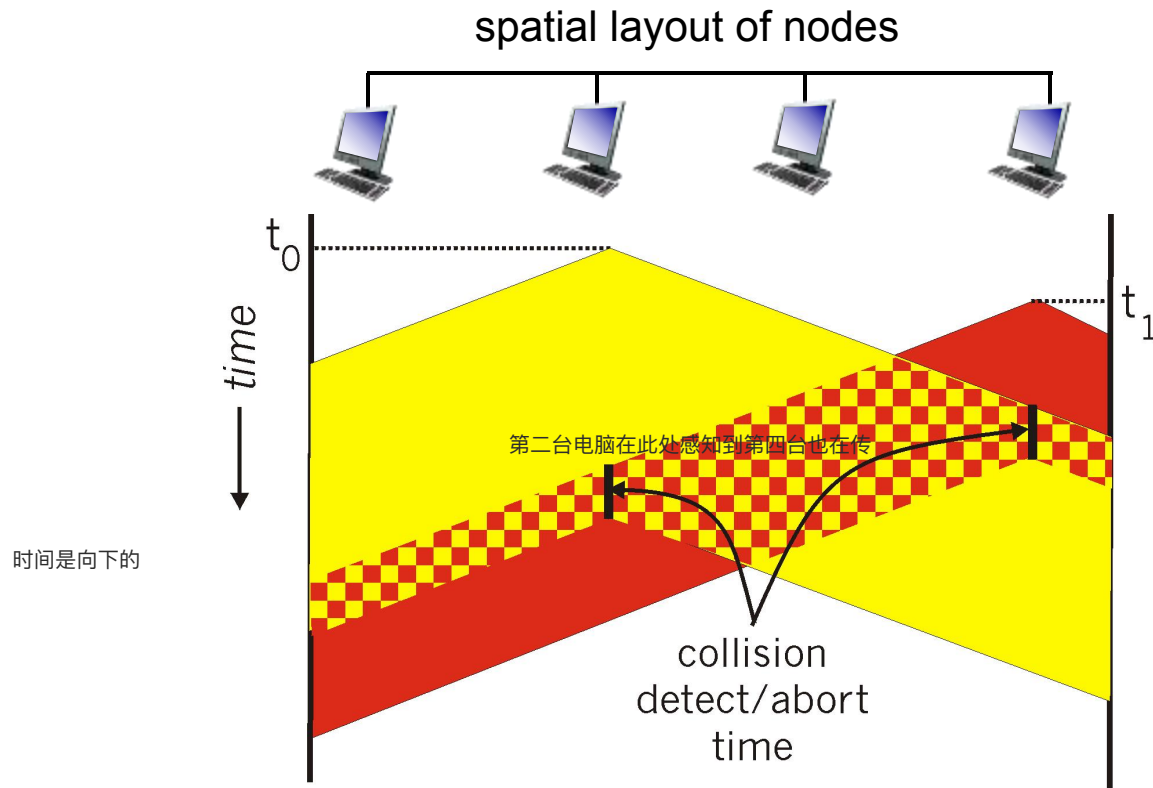
如果不一致/幅度异常 → 说明有人也在发 → 碰撞

无线时你自己在发射的信号非常强, 会把你接收的信号“淹没”:

本机发射功率 >> 远端到达的信号

你很难一边发一边听清别人是否也在发
所以 Wi-Fi 不用 CD, 而用 CSMA/CA
(碰撞避免)。

CSMA/CD (collision detection)



Ethernet CSMA/CD algorithm

1. network adapter (network interface card, NIC) receives datagram from network layer, **creates frame**
2. If NIC senses channel idle, starts frame transmission. If NIC senses channel busy, waits until channel idle, then transmits.
3. If NIC transmits entire frame without detecting another transmission, NIC is done with frame!
4. If NIC detects another transmission while transmitting, **aborts and sends jam signal**
jam signal (干扰信号) 是干嘛的?
用来确保所有节点都知道发生了碰撞
防止有的节点“没听清楚”, 继续发
5. After aborting, NIC enters **binary (exponential) backoff:**
 - after m th collision, NIC chooses K at random from $\{0, 1, 2, \dots, 2^m - 1\}$.
 - NIC waits $K \cdot 512$ bit times, returns to Step 2
 - **longer backoff interval with more collisions**

“Taking turns” MAC protocols

channel partitioning MAC protocols:

- share channel *efficiently* and *fairly* at high load
- inefficient at low load: delay in channel access, $1/N$ bandwidth allocated even if only 1 active node!

random access MAC protocols

- efficient at low load: single node can fully utilize channel
- high load: collision overhead

ALOHA, CSMA, CSMA/CD

“taking turns” protocols

look for best of both worlds!

- when one node wants to transmit, it can send at rate R .
- when M nodes want to transmit, each can send at average rate R/M

“Taking turns” MAC protocols

master node “invites” slave nodes to transmit in turn
一个主节点 master 按固定顺序，一个一个“点名”从节点（slaves）：

polling:

- master node “invites” slave nodes to transmit in turn

master 发一个 poll（询问/邀请）给某个 slave

被点到的 slave 如果有数据，就发 data

然后 master 再去点下一个

- maximum number of frames

为了公平/控制延迟，协议一般会规定：

每次被点名的节点最多只能发有限数量的帧（比如最多发 1 帧或最多发一小批）

- concerns:

- polling overhead

concerns (担忧/缺点)

(1) polling overhead (轮询开销)

即使某些节点没数据，master 也得轮到它时发 poll (课件右下角那句)

master 必须轮询每个节点，不管它有没有帧要发

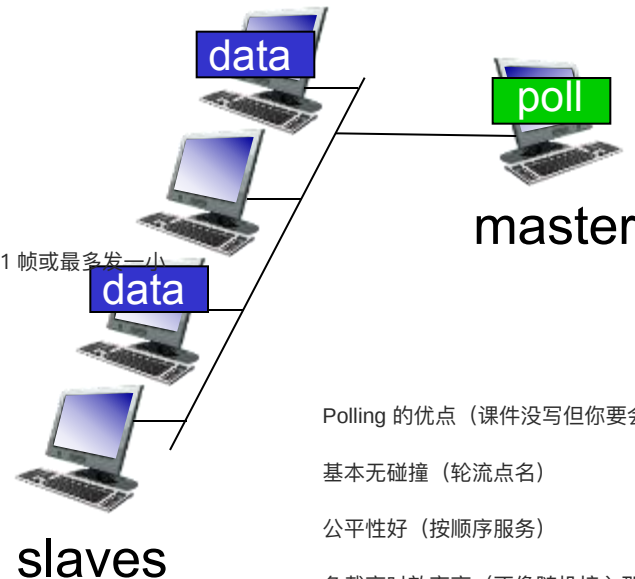
所以在低负载/很多空节点时，会浪费大量控制包（poll）→ 效率下降。

(2) single point of failure (单点故障 : master)

master 一旦挂了：

没人组织轮询

整个“轮流发”的机制就瘫痪
所以它有明显的中心化风险。



Polling 的优点（课件没写但你要会说）

基本无碰撞（轮流点名）

公平性好（按顺序服务）

负载高时效率高（不像随机接入那样一直撞）

The master node must poll each of the nodes in turn no matter they have frames or not

“Taking turns” MAC protocols

token passing:

token 按固定顺序在节点之间传递（像接力棒）

谁拿到 token，谁才有发送权

- control *token* passed from one node to next sequentially (fixed order).

- distributed 没有永远的master了

- concerns:

- token overhead

(1) token overhead (令牌开销)

- single point of failure (token)

token 本身也是控制信息：

即使没数据，token 也要不停在网络里转

这会占用一点带宽与处理开销

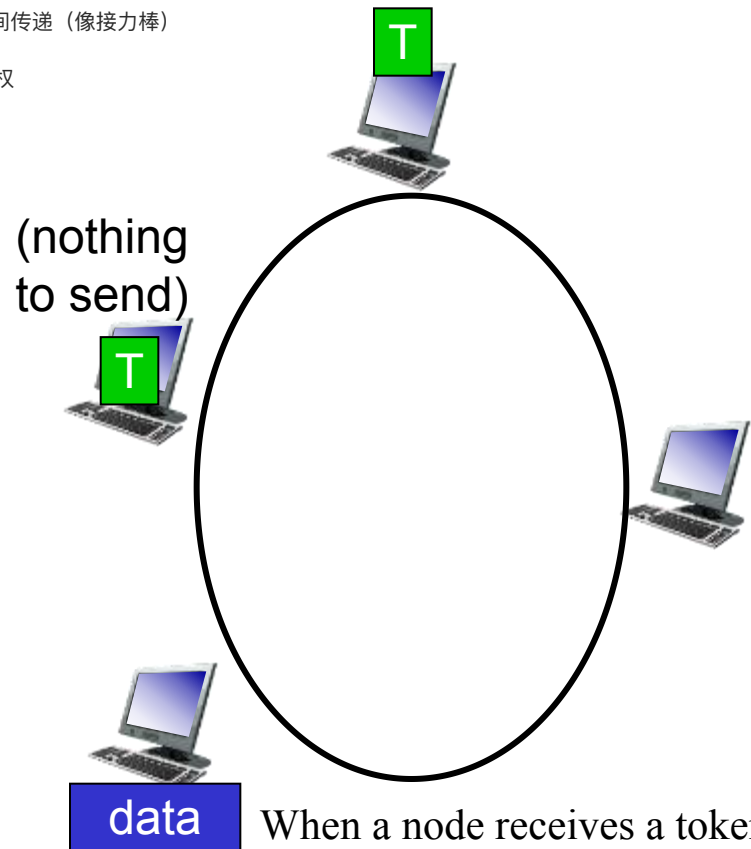
(2) single point of failure (单点故障：token)

这里的“单点”不是某台机器，而是“令牌机制”本身：

token 丢了/损坏 → 大家都不知道谁能发

token 卡住（某节点拿着不转发、死机）→ 网络阻塞

现实系统通常要有“token 丢失检测与重新生成”的机制，否则会挂死。



When a node receives a token,

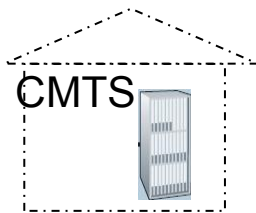
- if it has some frames to transmit, it holds onto the token;
- otherwise, it immediately forwards the token to the next node.

Case study: Cable access network

DOCSIS 是“有线宽带 (Cable Internet)”的标准/规范。

规定了：在有线电视的同轴电缆上，怎么传互联网数据（上下行怎么分、怎么共享信道等）

cable headend



MAP frame for
Interval $[t_1, t_2]$

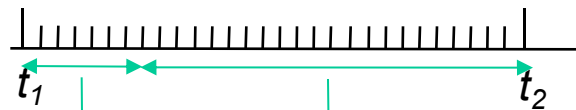
Downstream channel i

Upstream channel j



Residences with cable modems

DOCSIS 在上行中结合了随机接入和时分分配：
用户先通过竞争时隙随机请求资源，再由 CMTS
通过 MAP 帧分配无冲突的上行时隙，从而兼顾低
负载延迟和高负载效率。



Minislots containing
minislots request frames

Assigned minislots containing cable modem
upstream data frames

DOCSIS: data over cable service interface specification

■ FDM over upstream, downstream frequency channels

上行下行用不同的频段

■ Downstream (no multiple access); upstream (collision)

只有CMTS一个发送者

很多家庭同时向CMTS发

■ TDM upstream: some slots assigned, some have contention

但这些 minislot 分两类：

some slots assigned：有些时隙是“分配好
的”，指定某个用户用（不会冲突）

some have contention：有些时隙是“竞争
的”，大家可以抢（可能冲突）

• downstream MAP frame: assigns upstream slots

CMTS 会在“下行”广播一个 MAP 帧，用来告诉大家：接下来一段时间里，上行的每个 minislot 是怎么分配的

• request for upstream slots (and data) transmitted random access (binary backoff) in selected slots

用户想上传数据之前，通常得先“申请上行资源”（申请时隙）；这些 request 不是随便什么时候发，而是只能在 CMTS 指定的一些“竞争 minislot”里发（selected slots）。

因为这些 minislot 是开放抢的，所以属于 random access（随机接入）；如果很多人同时在竞争 minislot 里发 request，就会冲突。

冲突后怎么办？——用 二进制指数退避

Summary of MAC protocols

- *channel partitioning*, by time, frequency or code
 - Time Division, Frequency Division
 - *random access* (dynamic),
 - ALOHA, S-ALOHA, CSMA, CSMA/CD
 - carrier sensing: easy in some technologies (wire), hard in others (wireless)
 - CSMA/CD used in Ethernet
 - CSMA/CA used in 802.11
 - *taking turns*
 - polling from central site
 - token passing
1. CMTS 在下行广播 MAP (排班表) : 告诉大家哪些 minislot 是竞争用, 哪些 minislot 已经分配给谁
 2. 想上传的用户先在“竞争 minislot”里发 request (可能冲突, 冲突就 binary backoff 再来)
 3. CMTS 收到 request 后, 在后续 MAP 里给该用户分配“assigned minislot”
 4. 用户在被分配的 minislot 里发送真正的 upstream data (这时基本不会冲突)

Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs

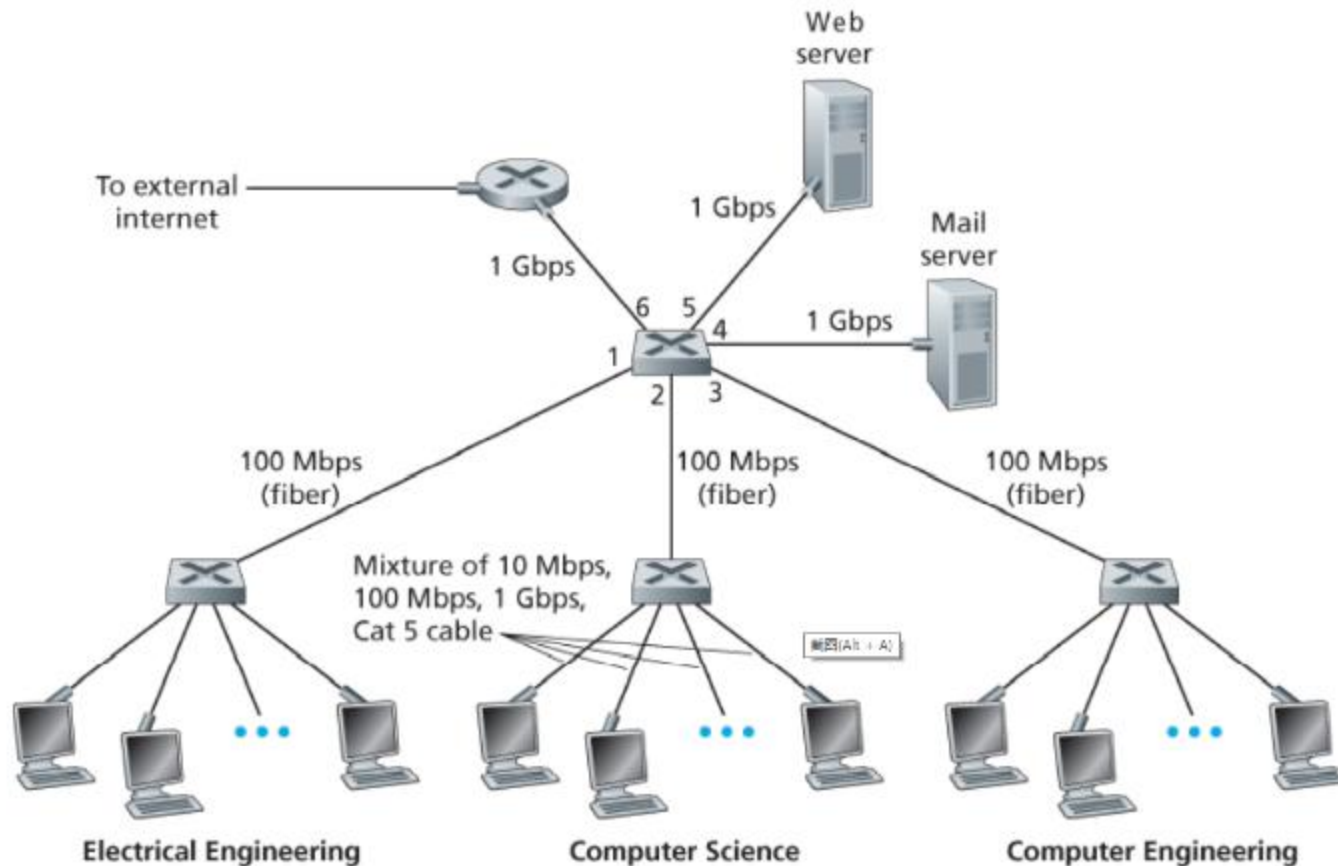
- addressing, ARP
- Ethernet
- switches
- VLANs

6.5 link virtualization: MPLS

6.6 data center networking

6.7 a day in the life of a web request

LANs



Because these switches operate at the link layer,

链路层看的東西是：以太网帧、MAC 地址

- don't recognize network-layer addresses
- don't use routing algorithms like RIP or OSPF to determine paths through switches

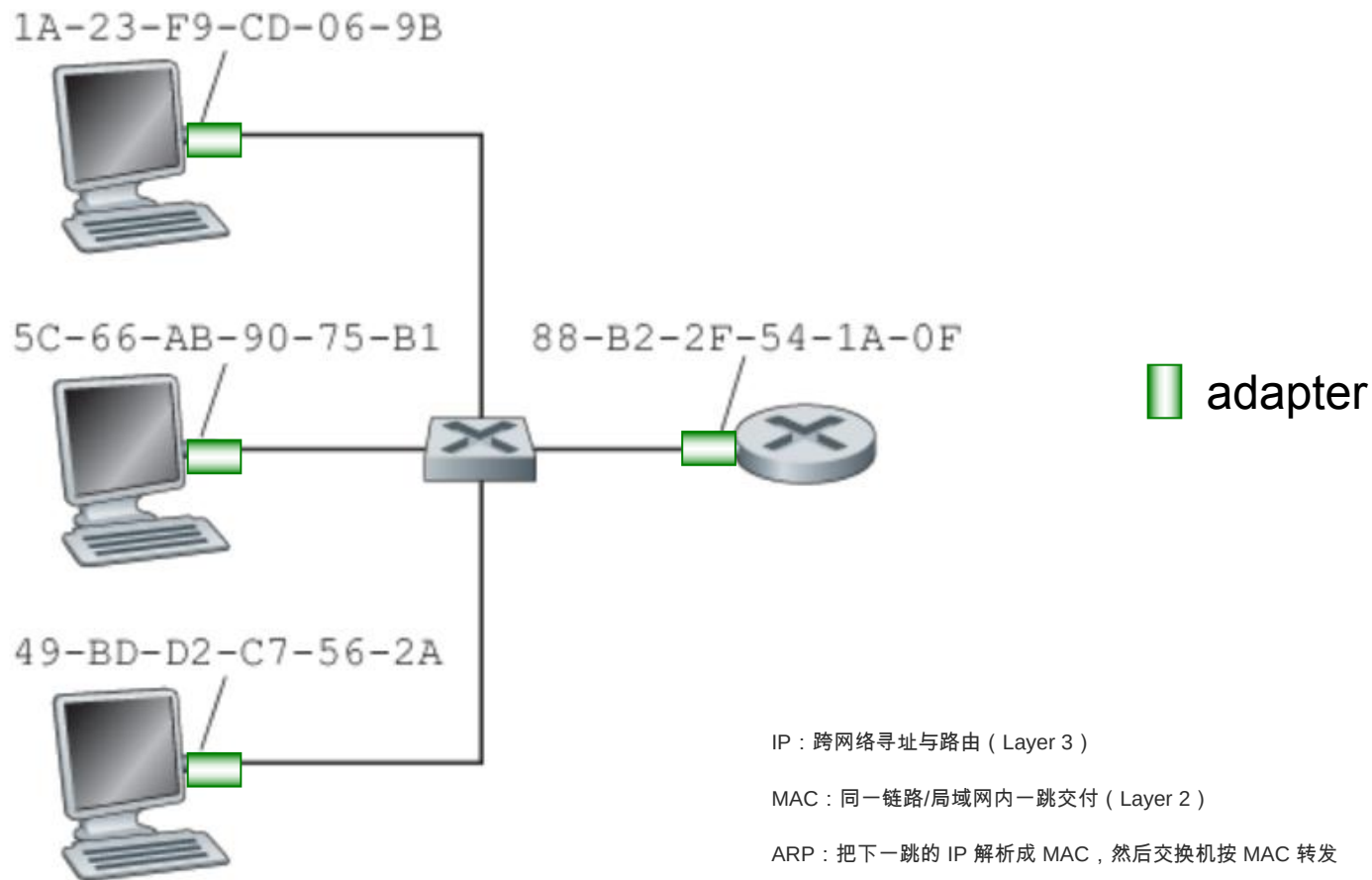
交换机=二层设备：只看 MAC，不看 IP；不跑路由协议，只靠 MAC 学习表转发

MAC addresses and ARP

- 32-bit IP address:
 - network-layer address for interface
 - used for **layer 3 (network layer) forwarding**
- MAC (or LAN or physical or Ethernet) address:
 - **Adapter (network interface)** rather than host or routers 交换机转发数据帧时，不需要“像主机一样”用自己的 MAC 当收发地址
 - Link-layer switches **do NOT have MAC addresses** 它只是中间搬运：进哪个端口、出哪个端口
 - function: *used “locally” to get frame from one interface to another physically-connected interface (same network, in IP-addressing sense)* MAC 只负责同一个局域网里“一跳”传输（把帧送到下一跳/同网段对方）
 - 48 bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable; no two adapters have the same address
 - e.g.: 1A-2F-BB-76-09-AD hexadecimal (base 16) notation (each “numeral” represents 4 bits)

LAN addresses and ARP

each adapter on LAN has unique *LAN* address



LAN addresses (more)

- MAC address allocation administered by IEEE
- manufacturer buys portion of MAC address space (to assure uniqueness)
- analogy:
 - MAC address: like ID Number
 - IP address: like postal address
- MAC flat address
 - can move LAN card from one LAN to another
- IP hierarchical address
 - address depends on IP subnet to which node is attached

LAN addresses (more)

- an adapter sends a frame to some destination adapter,
 - inserts the destination adapter's MAC address into the frame and then sends the frame into the LAN 这里写的是 MAC, 不是 IP
- an adapter receive a frame IP 在帧的“payload (载荷)”里
 - If there is a **match**, extracts the enclosed datagram and passes the datagram up the protocol stack;
 - If there **isn't a match**, discards 以太网里所有设备“都能看到”帧, 但只有目标设备会“接收处理”
- MAC broadcast address FF-FF-FF-FF-FF-FF
 - 发给局域网里的所有网卡
 - 看到这个目的 MAC
 - 都会当作“匹配”来处理

Questions

- How to determine interface's MAC address, knowing its IP address?
- How to send a datagram from one host to another?

ARP: address resolution protocol

ARP = 把 IP 地址解析成 MAC 地址

Question: how to determine interface's MAC address, knowing its IP address?

ARP: IP → MAC

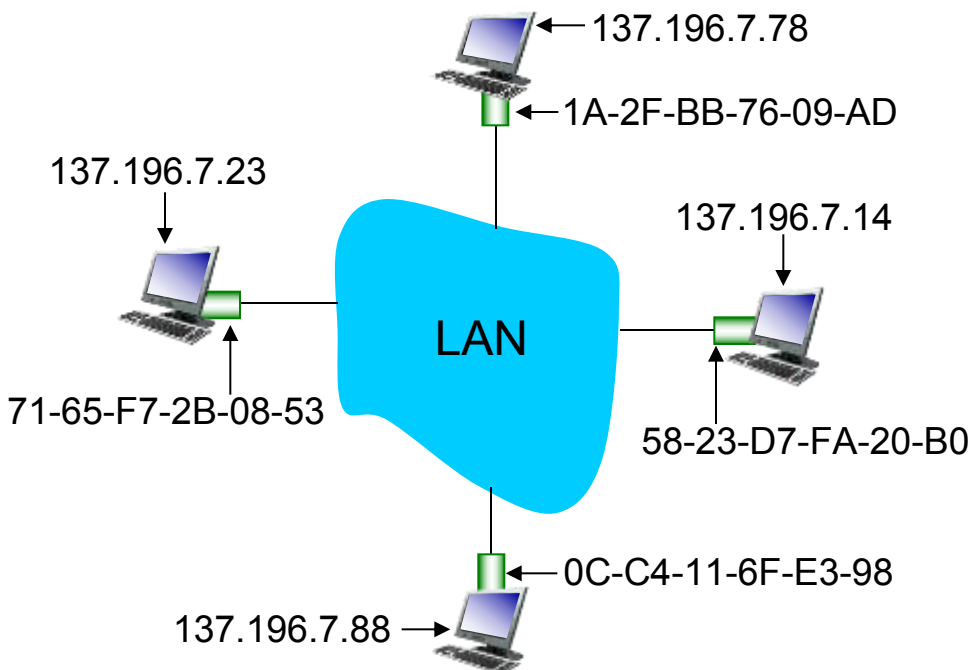
- Resolve addresses **only for interfaces on the same subnet**

ARP table: **each IP node (host, router) on LAN has table**

- IP/MAC address mappings for some LAN nodes:

< IP address; MAC address; TTL >

- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)



ARP protocol: same LAN

Host A wants to send datagram to host B (same subnet)

- B's MAC address not in A's ARP table.

Step 1: A **broadcasts** ARP query packet, containing B's IP address

- ARP packet: sending IP and MAC, receiving IP and MAC
- **destination MAC address = FF-FF-FF-FF-FF-FF**
- all nodes on LAN receive ARP query

Step 2: B receives ARP packet, replies to A with its (B's) MAC address

- frame sent to A's MAC address **(unicast)**

A caches (saves) IP-to-MAC address pair in its ARP table until information becomes old (times out)

ARP is “plug-and-play”: nodes create their ARP tables *without intervention from net administrator*

不需要管理员手动配置

不需要提前写映射表

主机一插网线:

ARP 自动工作

自动学习 IP↔MAC

Questions

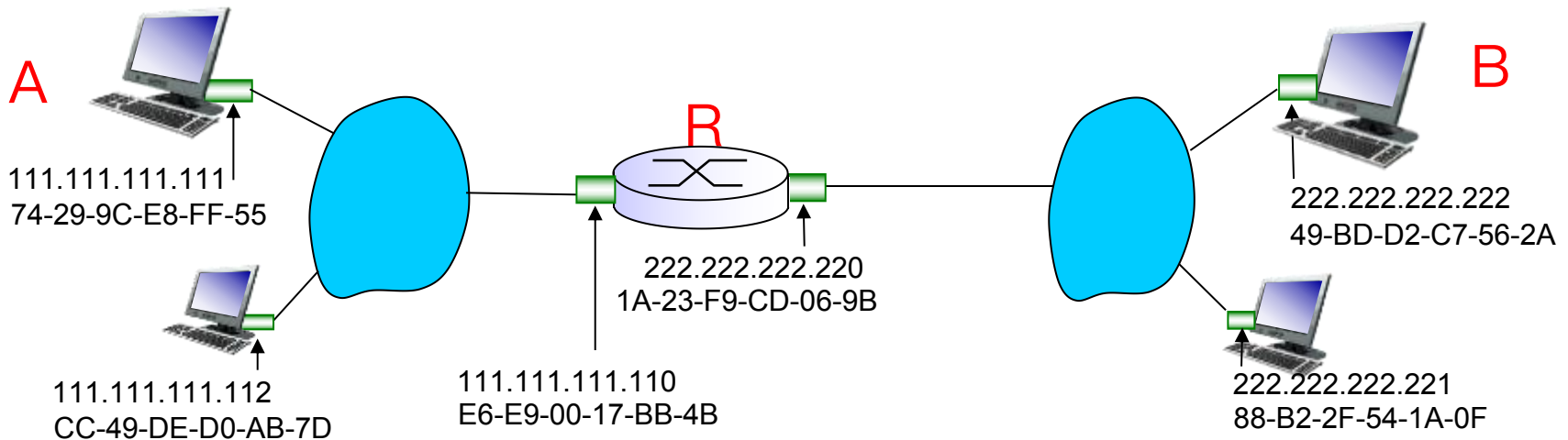
- How to determine interface's MAC address, knowing its IP address?
- How to send a datagram from one host to another?
 - Same subnet: framing with destination MAC; send it
 - Different subnets

Addressing: routing to another LAN

walkthrough: send datagram from A to B via R

- focus on addressing – at IP (datagram) and MAC layer (frame)
- assume A knows B's IP address
- assume A knows IP address of first hop router, R (how?)
通常是默认网关, DHCP 发的或手动配的
- assume A knows R's MAC address (how?)

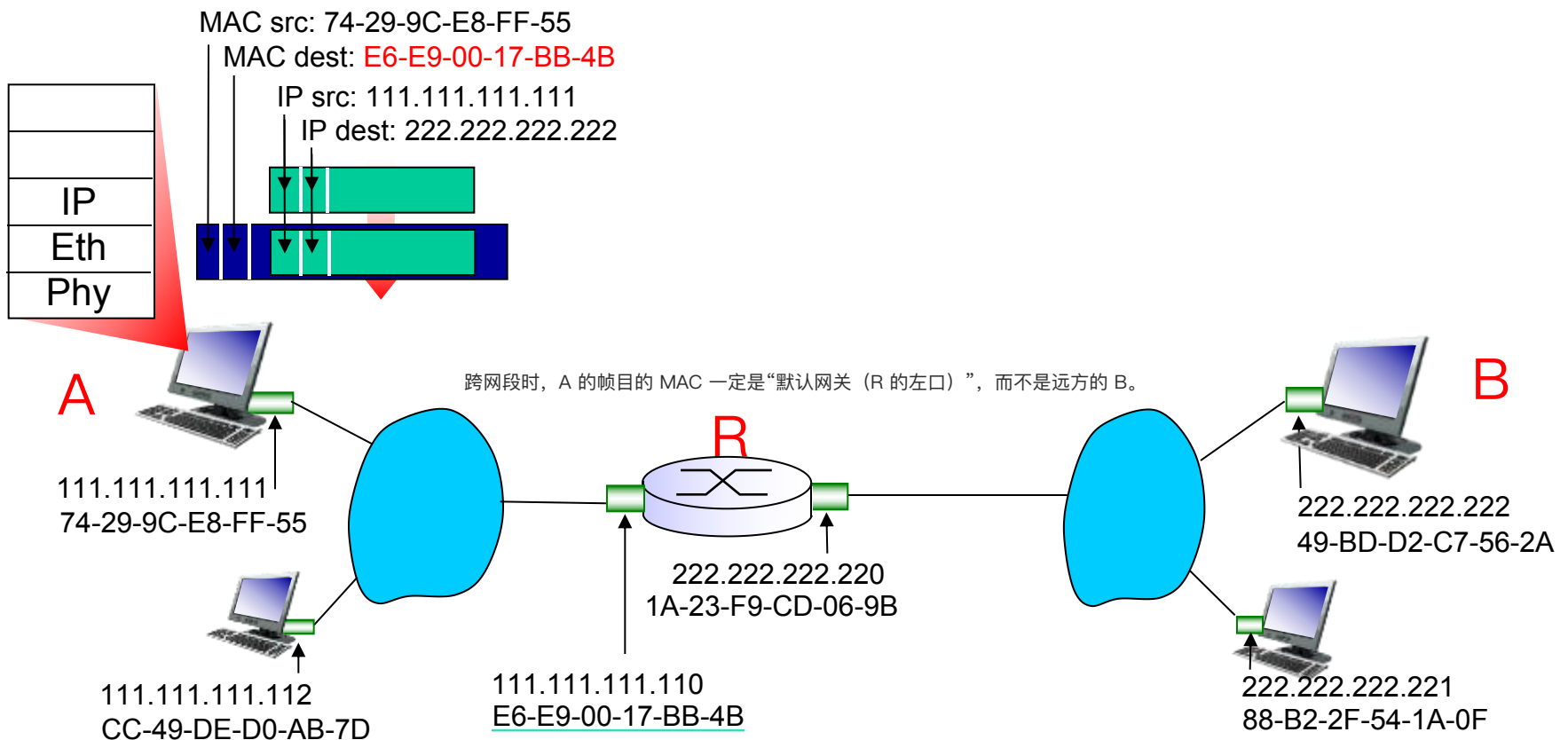
对 R 的 IP 做一次 ARP 得到



Addressing: routing to another LAN

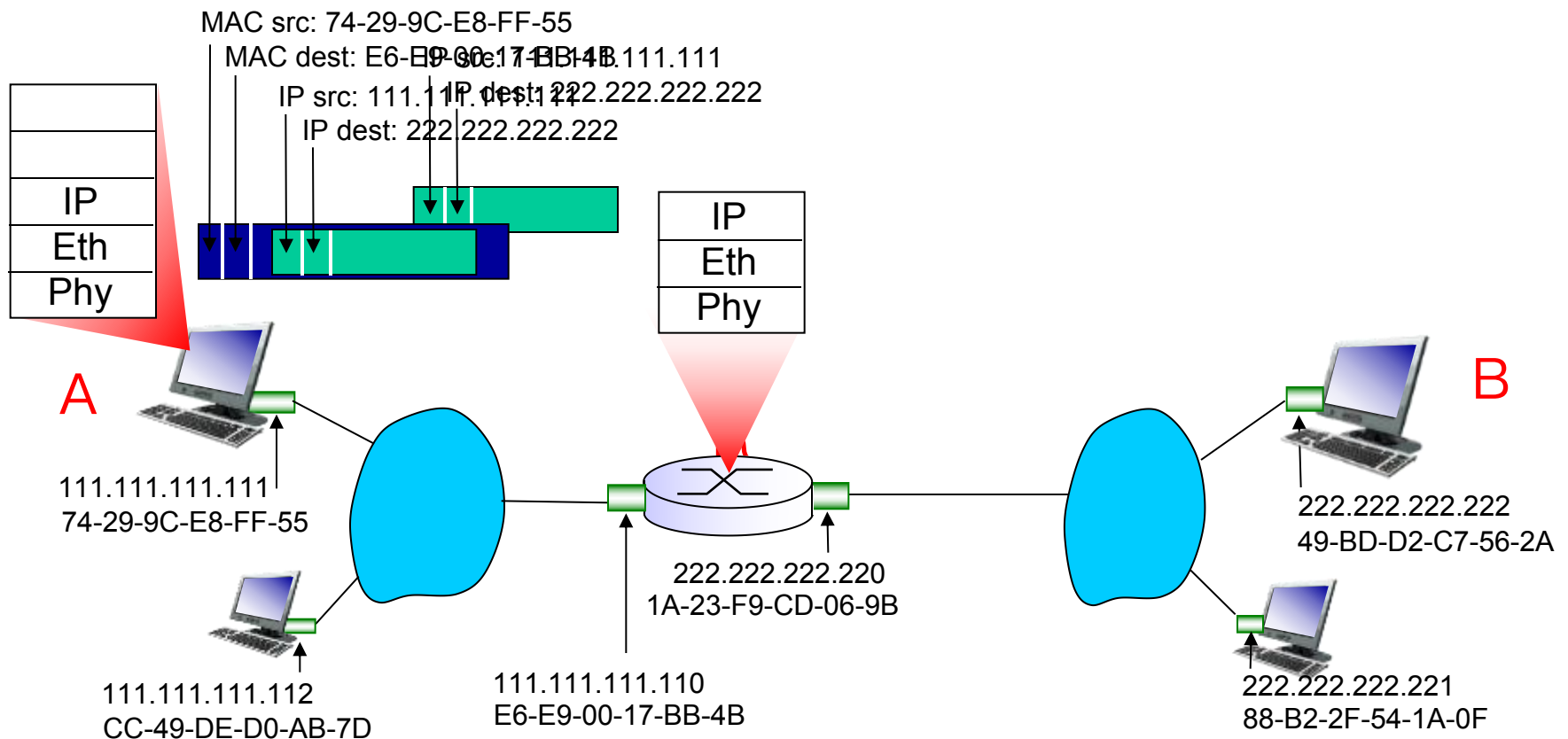
- A creates IP datagram with IP source A, destination B
- A creates link-layer frame with R's MAC address as destination address, frame contains A-to-B IP datagram

A 发给 R: 帧的目的 MAC 是 R, 不是 B



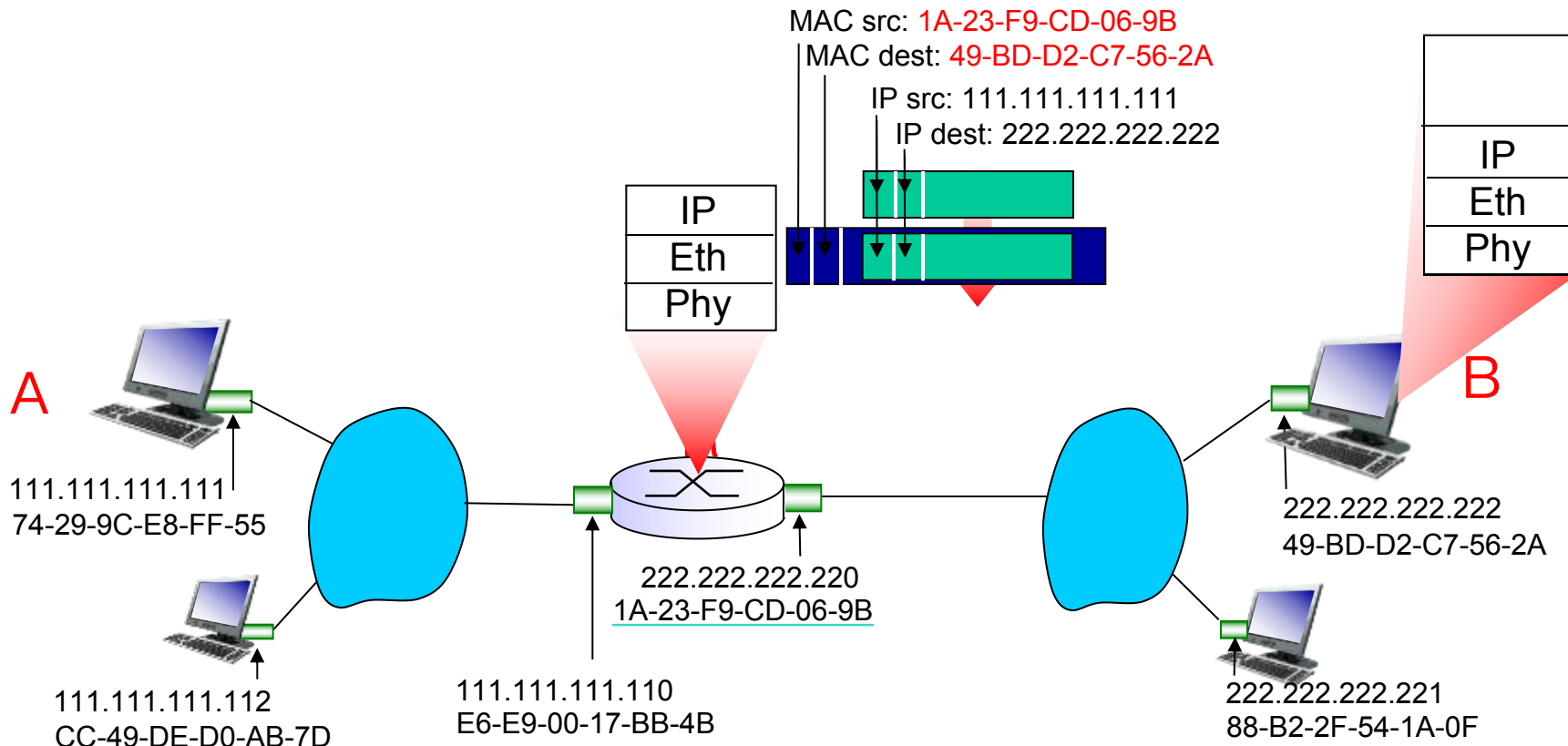
Addressing: routing to another LAN

- frame sent from A to R
- frame received at R, datagram removed, passed up to IP



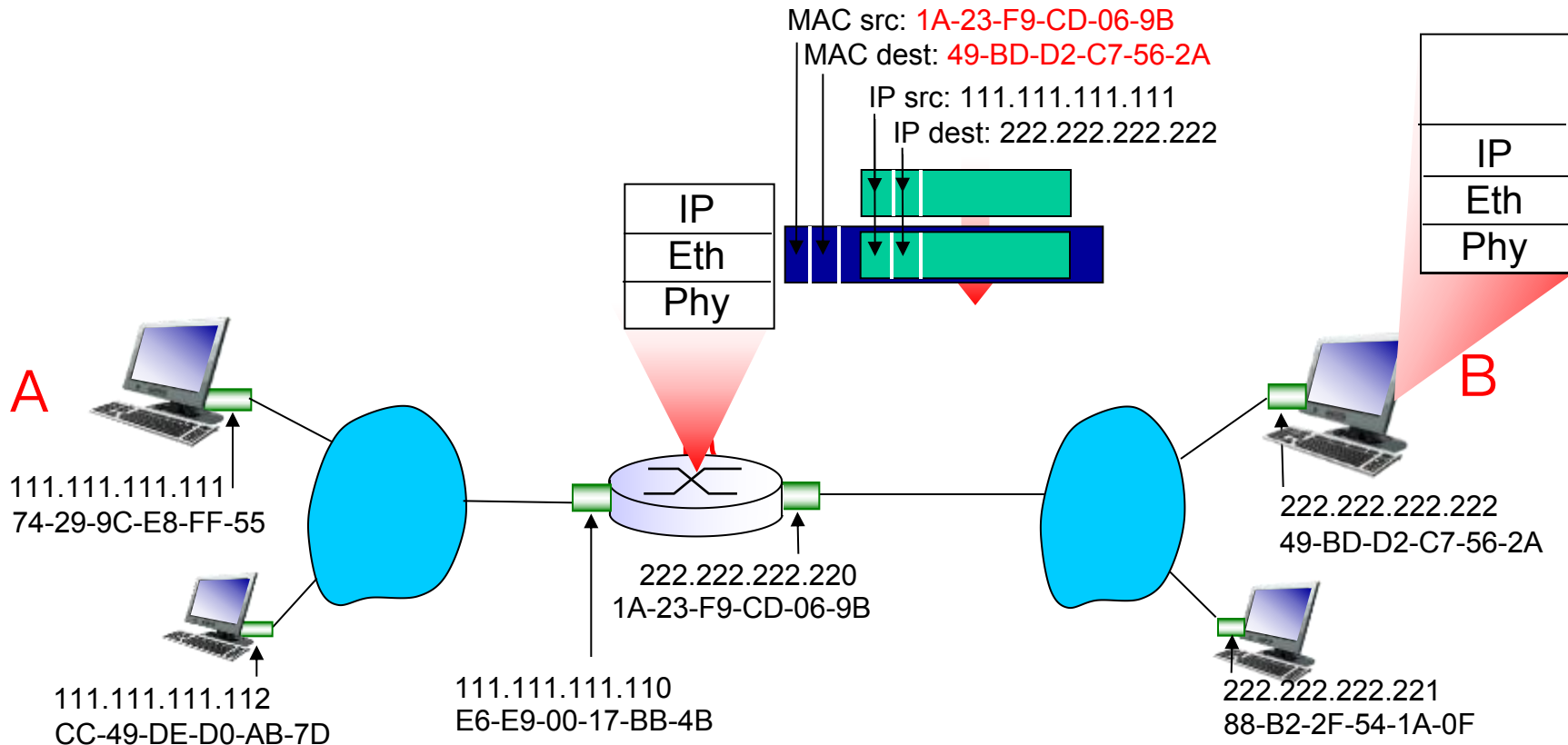
Addressing: routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as destination address, frame contains A-to-B IP datagram



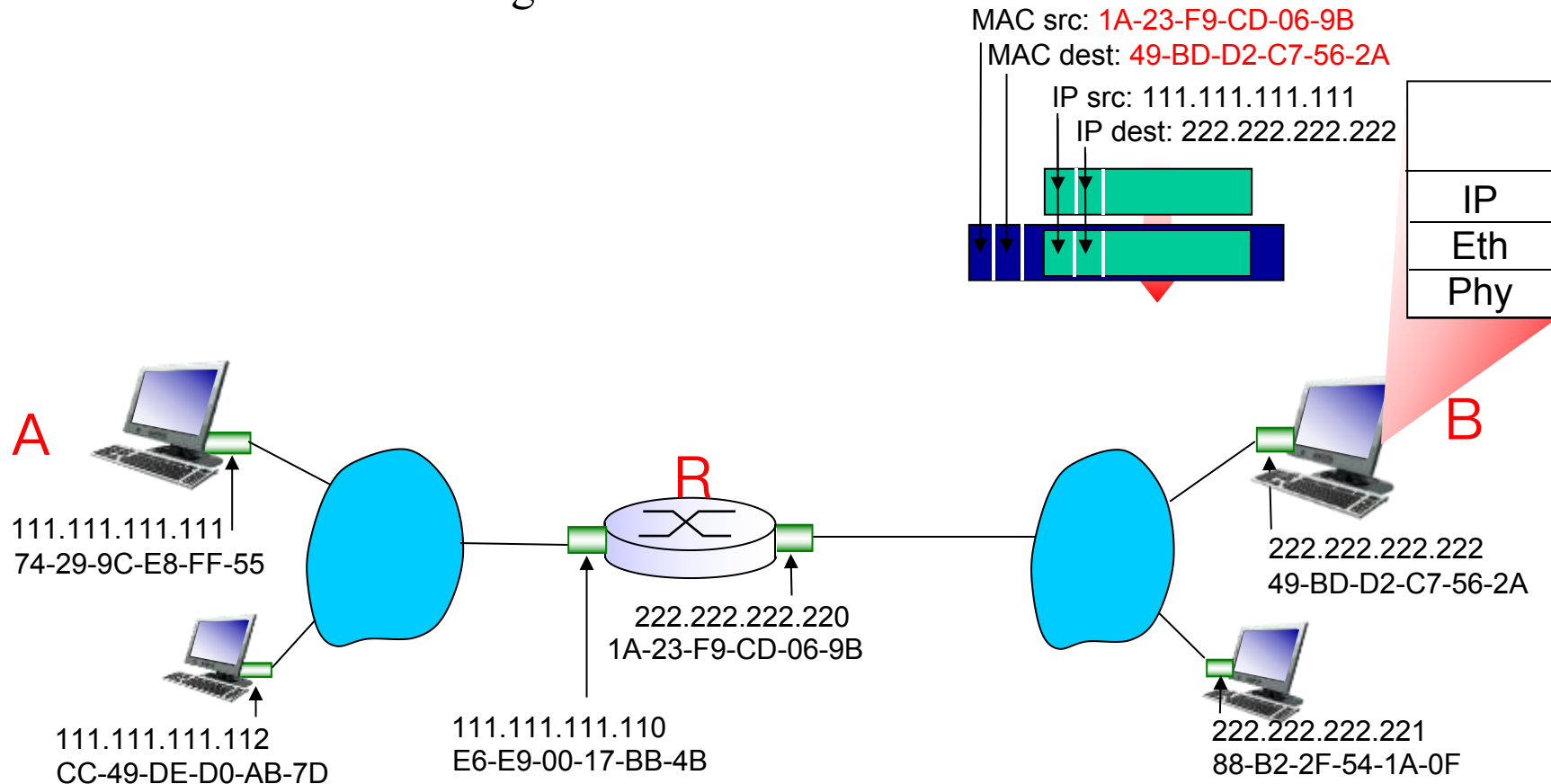
Addressing: routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as destination address, frame contains A-to-B IP datagram



Addressing: routing to another LAN

- R forwards datagram with IP source A, destination B
- R creates link-layer frame with B's MAC address as dest, frame contains A-to-B IP datagram



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/